

Capítol 1

Introducció

El projecte “Desenvolupament aplicació mòbil per una empresa de senyalització digital” correspon al meu Treball de Fi de Grau per a concloure els meus estudis en Enginyeria Informàtica, impartit a la Facultat d’Informàtica de Barcelona, Universitat Politècnica de Catalunya. Aquest treball pertany a l’especialització d’Enginyeria del Software, de manera que pretén abordar un seguit de conceptes treballats i adquirits en aquesta menció. Es desenvolupa en la modalitat B amb un conveni de Cooperació Educativa a Waapiti, dirigit per Daniel Novoa Gómez i assessorat per Carles Farré Tost (Departament d’Enginyeria de Serveis i Sistemes d’Informació).

L’objectiu principal d’aquest treball és realitzar un desenvolupament evolutiu d’un nou servei que complementa el sistema actual de Waapiti. Aquest servei complementari permetrà als *partners* gestionar els seus *players* de forma centralitzada, així com també proporcionar notificacions *push* a l’instant en cas d’incidències.

1.1 Contextualització

La senyalització digital [1], o també coneguda com cartelleria digital dinàmica, es refereix a l’ús de continguts digitals emesos a través de qualsevol suport audiovisual, com podria ser monitors LCD, pantalles de plasma, panells de LED o projectors. Aquesta nova tecnologia pretén substituir els cartells tradicionals oferint una millora en la presentació i promoció de productes. L’objectiu és crear una xarxa de sistemes audiovisuals amb continguts diàriament actualitzats. Un exemple de cas d’ús podria ser una xarxa de pantalles per franquícies (supermercat). Des de la central es realitza la gestió i la distribució del contingut que surt a les pantalles de cada sucursal. Totes les pantalles poden mostrar el mateix contingut

o diferents. En comparació amb la cartelleria tradicional la despesa és menor, ja que la impressió de paper és innecessària i el sistema *Digital Signage* permet reaccionar amb més flexibilitat a les necessitats de cada negoci.

Waapiti és una empresa tecnològica que es dedica en aquest àmbit. Ofereix solucions de cartelleria digital amb una interfície intuïtiva que està dissenyada per ser compatible amb qualsevol aplicació. Disposa d'una plataforma escalable, basada en web i sense servidor; permetent així distribuir automàticament els continguts, entre altres funcionalitats destacables, com rols d'usuari multinivell, funcionament en segon pla, accions massives, integració de música de fons, etc. A part de distribuir contingut, també permet monitoritzar els *players*, configurar canals, crear programacions, gestió d'usuaris, etc. A més a més, l'empresa també ofereix suport d'integració de tercers, com per exemple: mesura d'audiència, segmentació de públic, sistema de cues, etc.

L'empresa té dues vies de negoci. D'una banda els clients finals, el qual actua com a consultoria per tal de satisfer les necessitats del client: tipus de contingut i tecnologia a implementar. D'altra banda, treballa amb *partners*, que essencialment són empreses de tercers que disposen dels seus propis clients i els hi ofereixen solucions de senyalització digital mitjançant la tecnologia de Waapiti.

Els usuaris finals d'aquest projecte són els *partners*.

1.2 Conceptes

En aquest apartat es detalla una llista de diferents terminologies que s'anirà esmentant al llarg del treball, amb el fi d'especificar el significat i evitar confusions al lector.

- **Players:** són equips informàtics que fan la funció d'intermediari entre la plataforma de Waapiti i el suport audiovisual en si, permetent així la sincronització i reproducció dels continguts dels clients. Aquests poden ser petits ordinadors, que s'instal·len als suports audiovisuals, o pantalles amb tecnologia *SoC (System on Chip)*[2], el qual ho porten integrat. Per tal d'establir aquesta comunicació l'empresa instal·la un software propi en aquests *players*, que està preparat per donar suport a múltiples tipologies de *players*.
- **Continguts:** qualsevol tipus de contingut audiovisual (audio, vídeo, imatge, animacions, plantilles, etc.). Els clients ho pugen a la plataforma i després poden programar-les per tal de ser reproduïts.
- **Partners:** empreses de tercers que ofereixen servei/consultoria de senyalització digital utilitzant la tecnologia de Waapiti. Disposen d'accés a la

plataforma i poden gestionar els continguts dels clients.

- **Locations:** fan referència a l'adreça física dels *players*, a la localització d'aquests.

1.3 Identificació del problema

Com s'ha mencionat a la contextualització, per tal de que hi hagi comunicació entre la tecnologia Waapiti i els suports audiovisuals es requereix la presència dels *players*.

El procés d'instal·lació d'un nou *player* per part d'un *partner* comença amb la pròpia instal·lació física d'aquest. Seguidament, per tal de registrar-ho a Waapiti s'ha de donar d'alta a la plataforma i actualment es disposa de dues vies:

- **Manual:** el *partner* ha de registrar el *player* a la plataforma manualment, introduint les dades dels diferents camps que es demanen. En aquest punt entra en joc l'error humà ja que el registre consisteix en introduir una combinació de caràcters (número de sèrie del *player*) de longitud considerablement alt, a més a més del tipus i configuració d'aquesta.
- **Servei tècnic:** el *partner* truca al servei tècnic de Waapiti i ho gestiona. Aquesta via implica disposar hora d'oficina per part de l'empresa i, depenent de la zona horària del *partner*, això pot resultar inviable o complicat de coincidir.

En altres paraules, és un procés laboriós i molt manual.

A més, una de les necessitats d'un *partner* és conèixer els estats dels *players* dels seus clients i de qualsevol incidència. Actualment, aquestes notificacions són sota demanda, és a dir, el *partner* ha d'accedir a la plataforma i revisar un per un si tot està funcionant correctament, o mitjançant trucada per part del servei tècnic de l'empresa.

Així doncs, es requereix l'anàlisi, disseny i desenvolupament d'un nou servei que complementi el sistema actual. D'aquesta manera, s'aconseguirà augmentar la completesa del sistema, així com, millorar l'experiència d'usuari i el servei que se li ofereix als *partners*.

1.4 Actors implicats

Els actors implicats són totes aquelles parts interessades, beneficiades i implicades en el projecte. Són de vital importància, ja que ajuden a definir correctament el

projecte. Per tant, hem de tenir en compte les seves opinions, exigències i requisits. A continuació es llisten els diferents actors implicats del projecte.

Partners

Com ja s'ha mencionat, són empreses de tercers que ofereixen consultoria de cartelleria digital mitjançant la tecnologia de Waapiti. Són els usuaris finals d'aquest projecte.

Waapiti

Waapiti és l'empresa que ofereix la tecnologia de cartelleria digital. Podem diferenciar diferents actors:

- ***Product Owner:*** és l'encarregat de maximitzar el valor del producte entregat i el retorn de la inversió. És el representant dels clients dins de l'equip, la seva veu representa les parts interessades internes i és responsable d'entregar el valor més alt possible al negoci.
- **Servei Assistència Tècnica (SAT):** és el departament de manteniment i atenció al client. Amb l'aplicació es pretén facilitar la gestió de les incidències dels *players*.
- **Director Treball de Fi de Grau:** Daniel Novoa Gómez serà la persona que supervisarà i guiarà el projecte perquè avanci en la direcció correcta.
- **Desenvolupador:** és l'encarregat d'implementar tot el software, documentar el projecte i assegurar el correcte funcionament. Només hi ha un i és l'autor d'aquest Treball de Fi de Grau.

Capítol 7

Especificació de requisits

En aquest capítol es descriuen els requisits que ha de complir el sistema a desenvolupar. Els requisits d'un projecte són aquelles característiques, expectatives o capacitats que ha de tenir un producte per satisfer les necessitats de negoci.

Els requisits es dividiran en dos grups: requisits funcionals i no funcionals.

7.1 Requisits funcionals

Els requisits funcionals són aquells que defineixen les funcionalitats del sistema i/o estableixen els comportaments del software.

7.1.1 Històries d'usuari

Una història d'usuari [14] és una explicació general i informal d'una funcionalitat de software escrita des de la perspectiva d'usuari final. Té l'objectiu d'articular com un element, en aquest cas una funcionalitat, aporta valor al client. Les històries d'usuari segueixen el següent patró:

***Com** a [Rol], **vull** poder [Acció] ... **per** tal de [Benefici]*

On cada element entre claudators és un camp que s'ha de completar i responen a les següents preguntes:

- **[Rol]**: Per qui es desenvolupa aquesta funcionalitat? Es detalla el rol que té l'usuari en el sistema. Aquest rol pot ser un usuari final, un administrador, un supervisor, etc.

- **[Acció]:** Què intenta aconseguir l'usuari? Es detalla quina és l'acció que vol realitzar l'usuari. Aquesta acció pot ser una consulta, una modificació, una inserció, etc.
- **[Benefici]:** Quin benefici obtindrà després de realitzar l'acció? Es detalla quin és el benefici que assolirà l'usuari després de dur a terme l'acció definida a [Acció].

Per cada història d'usuari s'ha de definir el seu *Acceptance Criteria* (criteris d'acceptació), que és un conjunt de condicions que han de ser complides per tal que la història d'usuari es consideri finalitzada.

Gestió de clients

H1: Consultar llistat de clients	
Com a	<i>Partner</i>
Vull poder	Consultar el llistat dels meus clients
Per tal de	Poder consultar informació i/o seleccionar per gestionar les localitzacions
Criteris d'acceptació	• Es mostra un llistat de clients amb les dades bàsiques • L'usuari pot filtrar el llistat per diferents criteris i també ordenar-lo. • L'usuari pot seleccionar un client i consultar el llistat de localitzacions.

Taula 7.1: Història d'usuari 1

H2: Consultar llistat de localitzacions d'un client	
Com a	<i>Partner</i>
Vull poder	Consultar el llistat de localitzacions d'un client que he seleccionat prèviament
Per tal de	Poder consultar localitzacions i diferents <i>players</i>
Criteris d'acceptació	• Es mostra un llistat de localitzacions amb els seus respectius <i>players</i> • L'usuari pot filtrar el llistat per diferents criteris • L'usuari pot seleccionar un <i>player</i> i accedir a la vista de gestió de <i>players</i>

Taula 7.2: Història d'usuari 2

H3: Crear nova localització per un client	
Com a	<i>Partner</i>
Vull poder	Crear una nova localització a un client
Per tal de	Poder registrar un nou <i>player</i> en una nova localització
Criteris d'acceptació	• Es mostra un formulari per tal de crear una nova localització • S'utilitza l'API de Google Maps per tal de facilitar la búsqueda d'adreça, facilitant així la feina a l'hora d'omplir camps de formulari

Taula 7.3: Història d'usuari 3

Gestió de *players*

H4: Registrar nou <i>player</i> via lector codi QR	
Com a	<i>Partner</i>
Vull poder	Iniciar el procés de registre d'un <i>player</i> escanejant el codi QR que es mostra en la pantalla
Per tal de	Facilitar aquest procés
Criteris d'acceptació	• Si el codi QR s'ha escanejat bé i és vàlid, es navega a les següents pantalles per seleccionar client i localització • Una vegada seleccionats, l'usuari només haurà d'escollir la configuració del <i>player</i>

Taula 7.4: Història d'usuari 4

H5: Consultar estatus d'un <i>player</i>	
Com a	<i>Partner</i>
Vull poder	Consultar l'estatus actual dels diferents <i>players</i> dels meus clients
Per tal de	Poder assegurar-me el correcte funcionament d'aquests <i>players</i>
Criteris d'acceptació	• En el llistat de les localitzacions d'un client, a cada <i>player</i> es mostra la informació de l'estatus actual i la última connexió

Taula 7.5: Història d'usuari 5

H6: Realitzar accions sobre un <i>player</i>	
Com a	<i>Partner</i>
Vull poder	Realitzar diferents accions sobre un <i>player</i> en temps real
Per tal de	Poder oferir solucions als meus clients en cas d'incidència
Criteris d'acceptació	• En la pantalla d'un <i>player</i> es mostren botons per diferents accions: reiniciar <i>players</i>, apagar, forçar sincronització i reiniciar reproducció.

Taula 7.6: Història d'usuari 6

H7: Consultar informació general d'un <i>player</i>	
Com a	<i>Partner</i>
Vull poder	Visualitzar informació general d'un <i>player</i>
Per tal de	Poder assegurar-me el correcte funcionament
Criteris d'acceptació	• En la pestanya d'INFO d'una pantalla d'un <i>player</i> es mostren 'caixes' de diferents categories: general, configuració, previsualització i emmagatzematge

Taula 7.7: Història d'usuari 7

H8: Consultar informació avançada d'un <i>player</i>	
Com a	<i>Partner</i>
Vull poder	Visualitzar informació avançada d'un <i>player</i>
Per tal de	Poder assegurar-me el correcte funcionament i també realitzar accions sobre aquest
Criteris d'acceptació	• En la pestanya d'ADVANCED d'una pantalla d'un <i>player</i> es mostren 'caixes' de diferents categories: accions, pantalla i orientació, audio i sincronització. • Cada categoria conté informació i botons per realitzar canvis

Taula 7.8: Història d'usuari 8

H9: Consultar temps de línia d'un <i>player</i>	
Com a	<i>Partner</i>
Vull poder	Visualitzar el temps de línia d'un <i>player</i>
Per tal de	Poder assegurar-me el correcte funcionament
Criteris d'acceptació	• En la pestanya d'UPTIME d'una pantalla d'un <i>player</i> es mostra un calendari amb el temps de línia de cada dia per hores.

Taula 7.9: Història d'usuari 9

H10: Consultar historial de logs d'un <i>player</i>	
Com a	<i>Partner</i>
Vull poder	Consultar l'historial de logs d'un <i>player</i>
Per tal de	Poder assegurar-me el correcte funcionament
Criteris d'acceptació	• En la pestanya de PLAYLOGS d'una pantalla d'un <i>player</i> es mostra un llistat amb els logs de reproducció. • Cada log conté informació del contingut que s'ha reproduït • Via <i>sockets</i> es poden rebre els logs en temps real

Taula 7.10: Història d'usuari 10

H11: Editar un <i>player</i>	
Com a	<i>Partner</i>
Vull poder	Editar la configuració d'un <i>player</i>
Per tal de	Poder realitzar les modificacions necessàries i així satisfer els meus clients
Criteris d'acceptació	• En la pestanya d'INFO d'una pantalla d'un <i>player</i>, en la 'caixa' de general hi ha un botó per editar la configuració.

Taula 7.11: Història d'usuari 11

Notificacions *push*

H12: Rebre notificacions <i>push</i>	
Com a	<i>Partner</i>
Vull poder	Rebre notificacions <i>push</i>
Per tal de	Mantenir-me informat de qualsevol incidència
Criteris d'acceptació	• Si l'usuari té activat els permissos de notificacions, rebrà notificacions <i>push</i> en el seu dispositiu en cas d'incidències.

Taula 7.12: Història d'usuari 12

H13: Personalitzar notificacions <i>push</i>	
Com a	<i>Partner</i>
Vull poder	Personalitzar notificacions <i>push</i>
Per tal de	Centrar-me en les notificacions que m'interessin de veritat
Criteris d'acceptació	• L'usuari pot establir els tipus de notificacions que vol rebre en 3 nivells (general, per client i per <i>player</i>).

Taula 7.13: Història d'usuari 13

H14: Consultar llistat de notificacions <i>push</i> rebudes	
Com a	<i>Partner</i>
Vull poder	Consultar un llistat de les notificacions <i>push</i> rebudes
Per tal de	Mantenir-me al dia de les incidències
Criteris d'acceptació	• En la pestanya de NOTIFICACIONS, es mostra un llistat amb les notificacions rebudes ordenades cronològicament. • Es diferencien les notificacions llegides i no llegides. • Si l'usuari clica sobre una notificació, aquesta li porta a la pantalla del <i>player</i> corresponent

Taula 7.14: Història d'usuari 14

Gestió de perfil

H15: Editar perfil	
Com a	<i>Partner</i>
Vull poder	Editar informació del meu perfil
Per tal de	Satisfer les meves necessitats
Criteris d'acceptació	• L'usuari pot modificar informació del seu perfil, com el nom, la foto de perfil, etc

Taula 7.15: Història d'usuari 15

H16: Canviar contrasenya	
Com a	<i>Partner</i>
Vull poder	Canviar la meva contrasenya
Per tal de	Satisfer les meves necessitats
Criteris d'acceptació	• L'usuari pot modificar la seva contrasenya • L'usuari ha de confirmar la contrasenya actual per poder canviar • La contrasenya de ha de tenir un mínim de 8 caràcters, una lletra majúscula com a mínim, un número com a mínim i un caràcter especial

Taula 7.16: Història d'usuari 16

H17: Canviar idioma de l'aplicació	
Com a	<i>Partner</i>
Vull poder	Canviar l'idioma de l'aplicació
Per tal de	Satisfer les meves necessitats
Criteris d'acceptació	• L'usuari pot escollir entre: català, castellà, anglès, francès, italià, portuguès i alemany

Taula 7.17: Història d'usuari 17

7.2 Requisits no funcionals

En aquesta secció es descriuen els requisits no funcionals del sistema. Aquests especifiquen criteris que poden utilitzar-se per jutjar la operació d'un sistema en comptes dels seus comportaments específics. En altres paraules, no es refereixen directament a funcionalitats subministrades pel sistema, sinó a propietats del sistema, com podria ser el rendiment, la seguretat o la disponibilitat.

Requisit	<i>Accessibilitat i usabilitat</i>
Descripció	L'aplicació ha de ser senzill d'utilitzar per a qualsevol usuari.
Justificació	No podem garantir que tots els usuaris que utilitzin l'aplicació tinguin el mateix nivell d'ofimàtica, per tant, és crític que l'aplicació sigui senzill d'utilitzar. A més, una usabilitat complexa genera desinterès.
Condició de satisfacció	S'entendrà el requisit com a complet quan el feedback a usuaris de prova sigui bona

Taula 7.18: Taula RNF1

Requisit	<i>Fiabilitat i disponibilitat</i>
Descripció	L'aplicació ha d'estar sempre disponible per a l'usuari.
Justificació	Els <i>players</i> poden estar operatius totes les hores del dia, per tant, és necessari que el sistema també ho estigui per tal rebre qualsevol incidència. A més, un <i>partner</i> pot tenir clients de diferents zones horàries.
Condició de satisfacció	S'entendrà el requisit com a complet si el servei està disponible continuament.

Taula 7.19: Taula RNF2

Requisit	<i>Seguretat i privacitat</i>
Descripció	L'aplicació ha d'assegurar que les dades dels usuaris siguin confidencials i estiguin protegides dels accessos no autoritzats
Justificació	Al ser informació personal/confidencial, s'ha restringir accés a només als usuaris autoritzats
Condició de satisfacció	S'entendrà el requisit com a complet si les comunicacions entre el servidor i l'aplicació utilitzen el mínim d'informació possible i autenticat.

Taula 7.20: Taula RNF3

Requisit	<i>Multiplataforma</i>
Descripció	L'aplicació ha d'estar disponible per dispositius mòbils amb sistema iOS i Android
Justificació	Hem d'oferir compatibilitat pels dos sistemes ja que no podem garantir el sistema que utilitzen els nostres clients.
Condició de satisfacció	S'entendrà el requisit com a complet si l'aplicació està disponible pels dos sistemes i funciona correctament.

Taula 7.21: Taula RNF4

Requisit	<i>Internacionalització</i>
Descripció	L'aplicació ha de ser compatible amb diferents idiomes: anglès, francès, alemany, català, castellà, portuguès i italià
Justificació	L'empresa té <i>partners</i> arreu del món
Condició de satisfacció	S'entendrà el requisit com a complet si l'aplicació està disponible en els idiomes especificats anteriorment.

Taula 7.22: Taula RNF5

Requisit	<i>Imatge corporativa</i>
Descripció	L'aplicació ha de seguir la imatge corporativa de l'empresa
Justificació	Mantenir una estandarització dels productes de l'empresa. A més, intentar aportar la màxima familiaritat als nostres clients, que ja coneixen la plataforma.
Condició de satisfacció	S'entendrà el requisit com a complet si l'aplicació segueix la imatge de l'empresa.

Taula 7.23: Taula RNF6

Capítol 8

Arquitectura

En aquest capítol es descriu l'arquitectura de la solució, tant física com lògica. Es detalla com l'aplicació interactuarà amb el servidor de l'empresa, així com l'estructura del projecte per a que funcioni correctament.

8.1 Estructura bàsica

L'arquitectura física de la solució està format per un dispositiu mòbil, ja sigui sistema iOS o Android, i el servidor de l'empresa.

- **Dispositiu mòbil:** és el dispositiu on s'allotjarà l'aplicació. Es pot descarregar-se des de *Google Play Store*, en cas d'Android, i des de l'*App Store*, en cas d'iOS. Aquesta interactuarà amb l'API de Waapiti mitjançant peticions HTTP. Aquesta aplicació és el que es desenvolupa en aquest Treball de Fi de Grau.
- **Servidor Waapiti:** API [15] de l'empresa que consumeix l'aplicació. És el servidor que s'encarrega de comunicar-se amb la base de dades i altres serveis allotjats a AWS. [16]

Aquesta arquitectura segueix el model de 3 capes [17]:

- **Presentació:** és la capa que es comunica amb l'usuari. En aquest es mostra les dades del sistema i es recullen les dades introduïdes per l'usuari. En aquest cas, la capa de presentació és l'aplicació mòbil.
- **Domini:** és la capa que conté la lògica de negoci. En aquest es processa la informació i es realitzen les operacions necessàries per a que el sistema funcioni correctament.
- **Dades:** és la capa que conté la informació.

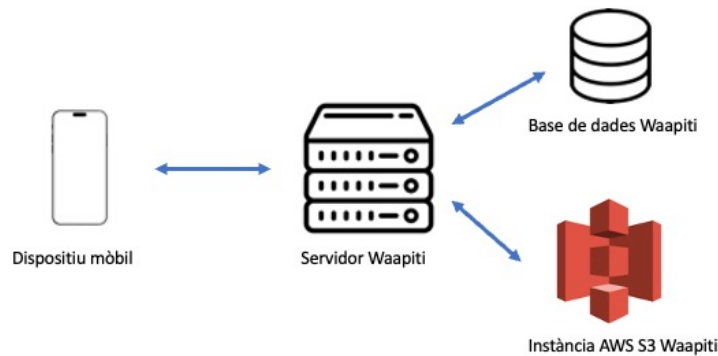


Figura 8.1: Diagrama arquitectura física

Font: Elaboració pròpia

8.2 Arquitectura lògica

8.2.1 Flux

En aquest projecte s'ha decidit desenvolupar utilitzant l'arquitectura Flux [18], introduïda per Facebook. És una arquitectura per al maneig i el flux de les dades en una aplicació web i té l'objectiu de substituir el patró *Model-View-Controller* (*MVC*).

El framework que s'utilitza en aquest treball és React Native, que corre sobre React (framework per crear interfícies frontend). React és una llibreria que està basada en components en cascada. Els components finals de la cascada realitzen la mateixa funció que les vistes en un patró MVC: crear la interfície gràfica i mantenir actualitzats els components renderitzats amb les dades.

Flux fa ús de *Controller-View*, és a dir, components de React que tenen l'únic objectiu de gestionar l'estat de l'aplicació. Aquests s'executen en cascada segons el flux d'execució. Flux proposa una arquitectura en el qual el flux de dades és unidireccional, obtenint així una depuració d'errors i escalabilitat més fàcil.

L'arquitectura està formada per 4 elements:

- **Store:** és el component que conté l'estat i la lògica de l'aplicació. Aquí es gestiona l'estat de múltiples objectes per un domini particular dintre de

l'aplicació. Al actualitzar-la, tots els components que estan consumint informació d'aquesta es veuen afectats. Seria lo més semblant al Model en un patró MVC. Cal destacar que no es pot modificar el contingut dels *stores* directament, sinó que s'ha de fer a través d'un *Dispatcher* i un *Action*.

- **Action:** és un objecte Javascript que indica una intenció de modificar l'estat de l'aplicació. Amb les dades obtingudes, el *Dispatcher* notifica els *Stores* que s'han de modificar, així actualitzant els components.
- **Dispatcher:** és l'eix central que gestiona tot el flux de dades. És el que notifica els *Stores* quan s'ha de modificar l'estat.

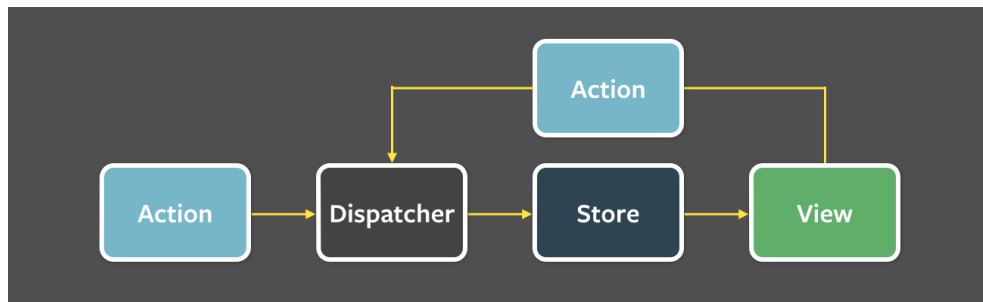


Figura 8.2: Diagrama Flux

Font: Documentació de Flux

8.2.2 Redux

Redux [19] és una llibreria que s'utilitza per a gestionar l'estat de l'aplicació. Està inspirada en l'arquitectura Flux, però amb una implementació més senzilla. En aquest treball s'utilitza Redux.

Redux es basa en tres principis:

- **Una sola font de veritat:** tot l'estat de l'aplicació està contingut en un únic Store. D'aquesta manera es facilita la depuració de l'aplicació.
- **L'estat és només de lectura:** l'estat només es pot modificar emetent una acció que indiqui els canvis.
- **Els canvis es fan amb funcions pures:** els *Reducers* són funcions pures que reben l'estat actual de l'aplicació i l'acció a realitzar i retornen un nou estat, sense modificar directament l'estat actual.

Una de les diferències més importants entre Flux i Redux és que en Redux no existeix el concepte de *Dispatcher*, ja que s'utilitzen funcions pures.

A més, Redux ofereix una API més familiar per a desenvolupadors de React, facilitant així la seva integració.

En la figura 8.3 podem veure un exemple del funcionament d'aquesta arquitectura representada en un diagrama de seqüència. En aquest cas, és un exemple de com es realitza un canvi d'estat en l'aplicació amb crides a l'API.

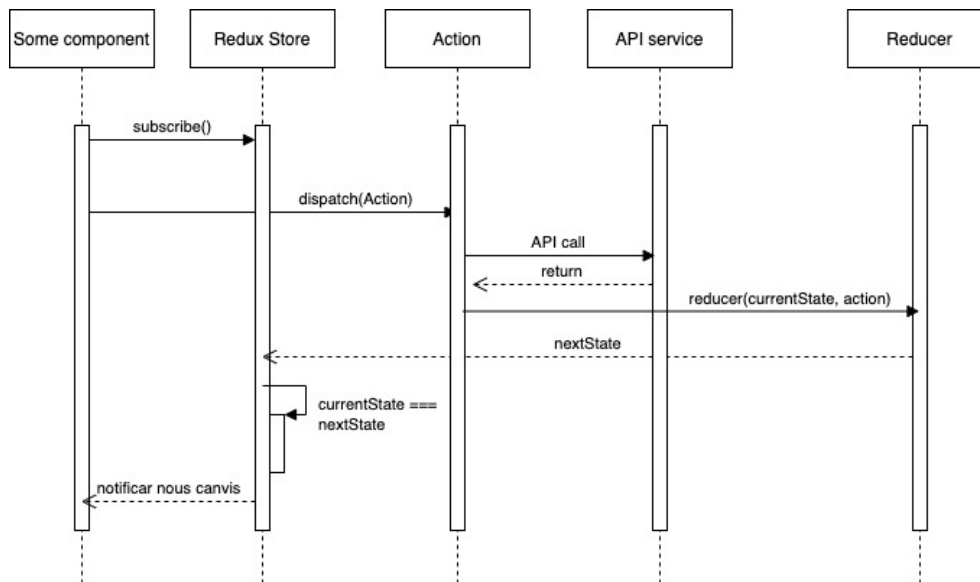


Figura 8.3: Diagrama de seqüència Redux
Font: Elaboració pròpia

8.3 Patrons de disseny

En aquesta secció es descriuen els patrons de disseny que s'han utilitzat en el desenvolupament del projecte.

8.3.1 Patró observador

El patró observador és un patró de disseny per observar l'estat d'un objecte en un programa. Aquest patró permet que un objecte, el subjecte, mantingui una llista d'objectes que observen el seu estat. Quan el subjecte canvia d'estat, es notifica als observadors, que poden reaccionar a aquest canvi. Tot objecte observador pot subscriure's o desubscriure's del subjecte en qualsevol moment.

Aquest patró es veu reflectit en l'arquitectura Flux. Els *stores* del sistema actuen com a subjecte.

8.3.2 Patró decorador

El patró decorador és un patró de disseny que permet afegir un comportament a un objecte, tant estàticament com dinàmicament.

En aquest projecte, s'ha utilitzat per facilitar les tasques de test. Els components estan embolicats amb un patró decorador per estendre les seves funcionalitats.

8.3.3 *Lazy Loading*

El concepte de *Lazy Loading* consisteix en carregar els objectes/components només quan són necessaris. D'aquesta manera, aconseguim optimitzar eficiència.

En aquest projecte, gràcies a la llibreria *React Navigation*, utilitzat per gestionar les navegacions de l'aplicació, s'ha implementat aquest concepte.

8.4 Diagrama model de dades

Per simplificar, només es mostren les entitats principals.

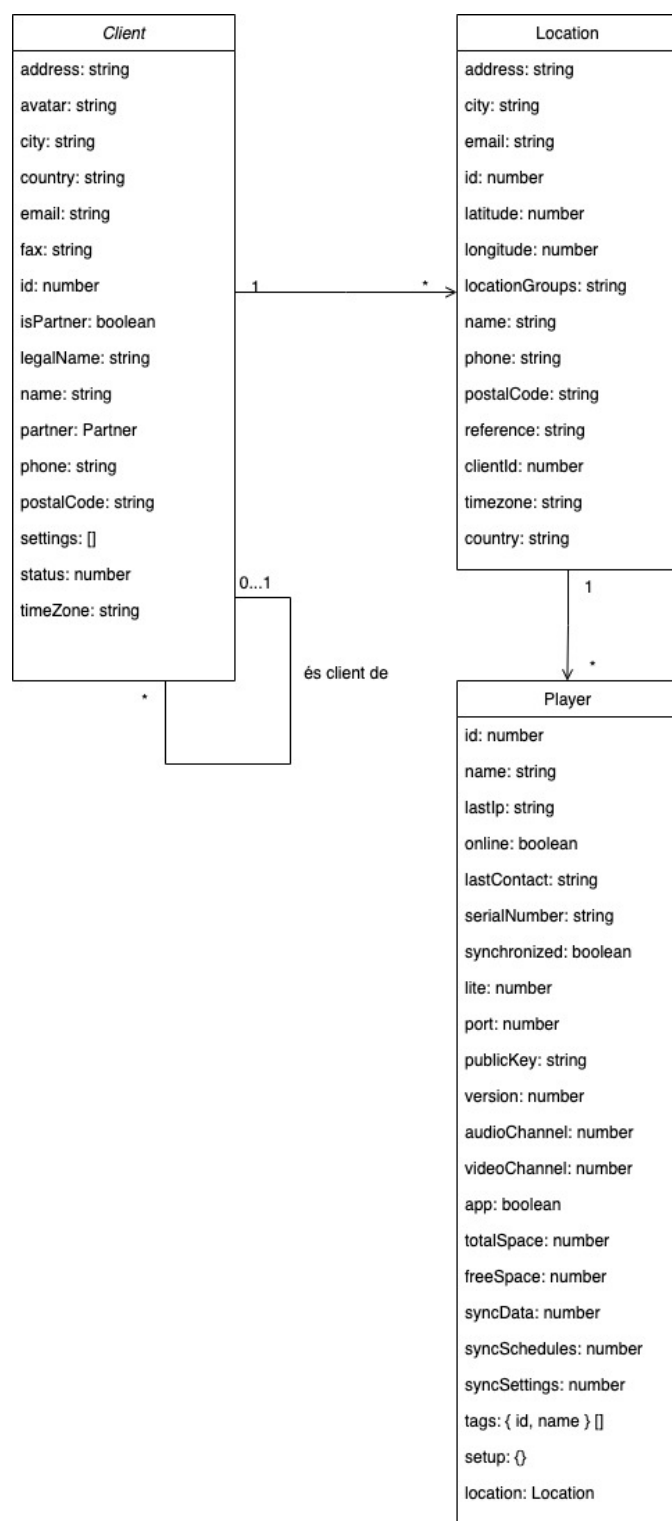


Figura 8.4: Model dades UML
Font: Elaboració pròpia

Capítol 9

Tecnologies

En aquest capítol es descriuen les tecnologies i els *frameworks*, així com les eines utilitzades durant el desenvolupament.

9.1 Tecnologies i *frameworks*

- **React Native:** és un marc de desenvolupament de codi obert creat per *Meta Platforms, Inc.* S'utilitza per desenvolupar aplicacions per Android, iOS, macOS, Web i Windows. Es basa en la llibreria de React oferint possibilitats d'incorporar codi natiu per a cada plataforma. Degut a la experiència prèvia que ja teníem amb React, ens va semblar una bona idea desenvolupar el projecte amb aquest *framework*.
- **React:** és una llibreria per construir interfícies d'usuari utilitzant Javascript.
- **Expo:** és un *framework* de codi obert per aplicacions natives d'Android, iOS i Web. Expo ofereix eines per facilitar el desenvolupament d'aquestes. Aquestes eines inclouen un entorn de desenvolupament que permet utilitzar tecnologies natives d'un dispositiu mòbil, instal·lar paquets de tercers utilitzant la pròpia comanda (`npx expo install`), servidor per compilar els codis, un emulador per a Android i iOS i un intèrprete de comandes.
- **Jest:** és un *framework* de testeig per Javascript. És també la llibreria recomanada per a projectes de React. [20]
- **NodeJS:** és un entorn d'execució de Javascript que permet executar-ho fora del navegador. En aquest cas s'utilitza per compilar.
- **TypeScript:** és un llenguatge de programació desenvolupat i mantingut

per *Microsoft*. És un superconjunt de Javascript, que essencialment afageix tipus i objectes basats en classes. [21]

- ***NestJS***: és un *framework* progressiu de NodeJS per crear aplicacions eficients, fiables i escalables de backend. [22]

9.2 Eines de desenvolupament

- ***Visual Studio Code***: és un editor de codi font desenvolupat per *Microsoft*.
- **GIT**: és un sistema de control de versions de codi font. Utilitzat per a la gestió de versions del codi font del projecte.
- **Dispositiu mòbil Android**: dispositiu per poder provar l'aplicació abans de ser publicada.
- **Dispositiu mòbil iOS**: dispositiu per poder provar l'aplicació abans de ser publicada.

Capítol 10

Implementació

En aquest capítol es descriu la implementació del sistema amb ajuda de suport visual (captures de pantalla). Els apartats estan agrupats segons la funcionalitat i seguint l'ordre de la navegació.

Com s'ha comentat en anteriors capítols, aquest Treball de Fi de Grau ha desenvolupat la part front-end de l'aplicació.

10.1 Autenticació i navegació

En aquest apartat s'explicarà com funciona l'autenticació en l'aplicació, així com la gestió de la sessió, inici de sessió biomètric i la navegació.

10.1.1 Cicle d'autenticació

En primer lloc, per a la navegació s'utilitza la llibreria *React Navigation* [23], que ofereix la transició entre pantalles de forma senzilla i un historial de navegació. Aquesta llibreria permet crear diferents piles (*stack*) de pantalles. Aquestes piles es poden anidar, de manera que es pugui navegar entre pantalles de diferents piles. L'aplicació està formada per dues piles principals: *AuthStack* i *AppStack*. La primera conté la pantalla d'inici de sessió i la segona conté totes les altres pantalles de l'aplicació, una pila per bloc de funcionalitat.

Per a la gestió de la sessió s'utilitza la llibreria *react-native-async-storage*, que ens permet guardar parells de clau-valor de manera local en els dispositius. En el nostre cas, guardem el token de l'usuari.

Una vegada explicada l'estructura, el cicle funciona de la següent manera:

1. Usuari obre l'aplicació.
2. L'aplicació comprova si hi ha un *token* guardat en local, utilitzant la llibreria *react-native-async-storage*.
3. Si no hi ha *token*, es mostra la pantalla d'inici de sessió, altrament es navega a l'aplicació.
4. Si l'usuari tanca la sessió, s'esborra el *token* de local i es navega a la pantalla d'inici de sessió.

En la figura 10.1 es pot veure el cicle d'autenticació de l'aplicació.

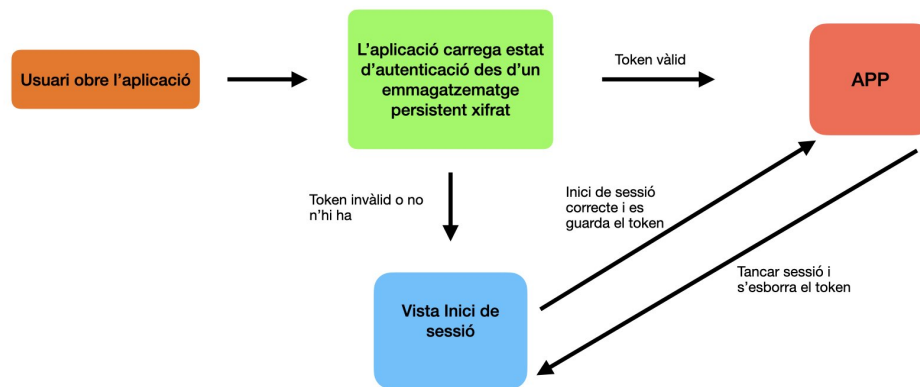


Figura 10.1: Cicle autenticació

Font: Elaboració pròpia

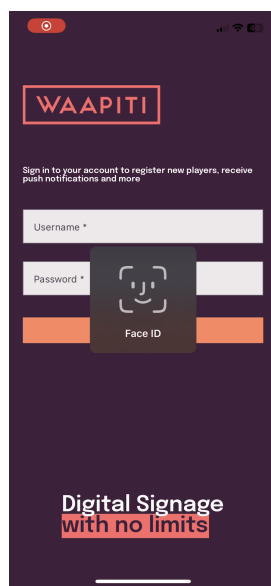
Crides a l'API

Descripció	Inici sessió amb usuari i contrasenya
Mètode	POST
URL	/auth/signin
Paràmetres	-
Cos de la petició	email: nom d'usuari password: contrasenya
Respostes	200: { id: number; displayName: string; avatar?: string; photoURL?: string; name: string; username: string; email: string; timezone: string; created?: string; online?: boolean; lastLogin?: string; role: string; type: string; client: Client; theme: {}; language: string; lastLanguage: string } 401: Unauthorized 404: Not Found

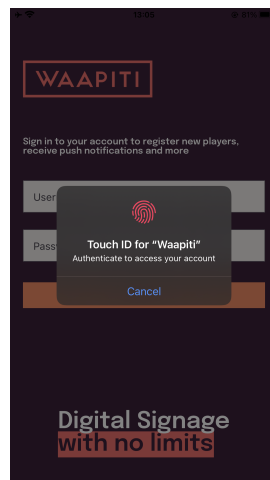
Descripció	Inici sessió amb <i>token</i>
Mètode	POST
URL	/auth/access-token
Headers	Authorization: Bearer token
Respostes	200: <pre>{ id: number; displayName: string; avatar?: string; photoURL?: string; name: string; username: string; email: string; timezone: string; created?: string; online?: boolean; lastLogin?: string; role: string; type: string; client: Client; theme: {}; language: string; lastLanguage: string }</pre> 401: Unauthorized 404: Not Found

Un altre punt a destacar sobre l'autenticació és que s'ha afegit l'inici de sessió utilitzant la tecnologia biomètrica [24]. Aquesta tecnologia permet l'usuari assegurar que és ell qui està iniciant sessió, ja que només pot fer-ho amb la seva cara o el seu dit. Cal mencionar que l'usuari pot sempre optar per no utilitzar aquesta tecnologia i iniciar sessió amb usuari i contrasenya de la plataforma. D'aquesta manera, afegim una capa més de seguretat a l'aplicació.

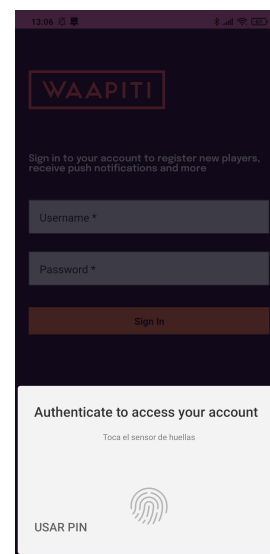
En la figura 10.2 es poden veure captures de pantalla de diferents dispositius amb l'inici de sessió biomètric: iPhone amb Face ID, iPhone amb Touch ID i dispositiu Android amb sensor de huella dactilar.



(a) Face ID



(b) Touch ID



(c) Huella dactilar

Figura 10.2: Autenticació biomètrica

10.1.2 Vista inici sessió

La vista de login segueix el mateix estil que la plataforma de Waapiti. Bàsicament està format per un formulari per introduir l'usuari i la contrasenya.

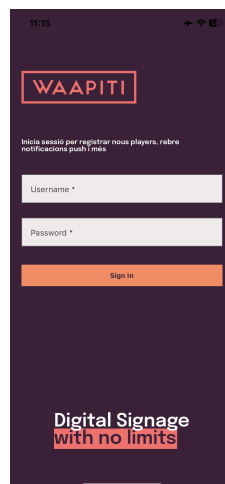


Figura 10.3: Vista inici de sessió

10.1.3 Navegació

Com s'ha comentat anteriorment, la llibreria utilitzada per a la navegació és *React Navigation*. S'ha optat per una navegació del tipus '*Bottom Tabs Navigator*', cada icona representa una funcionalitat principal amb les seves pantalles associades, en ordre d'esquerra a dreta:

- **Panell (*Dashboard*):** pantalla amb informació resumida dels dispositius i continguts dels clients.
- **Clients:** inclou llista de clients, llista de localitzacions i *players* d'un client i la gestió dels *players*.
- **Registrar nou *player*:** icona de més per registrar un nou *player*, aquí inclou la funcionalitat d'utilitzar la càmera del dispositiu per escanejar codi QR.
- **Notificacions:** pantalla amb totes les notificacions rebudes.
- **Perfil:** configuració de perfil.

En la figura 10.4 es pot veure una captura de pantalla de la navegació. La pila activa es mostra amb un color diferent a les altres.

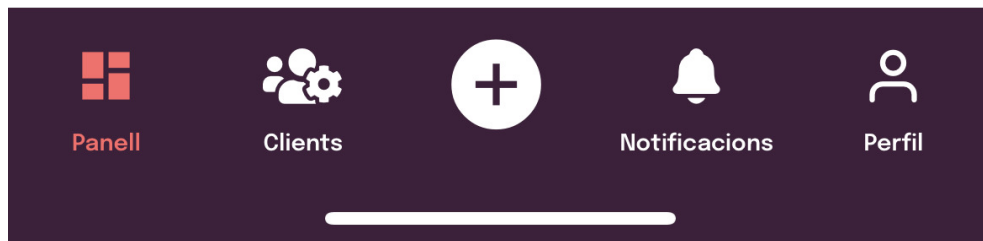


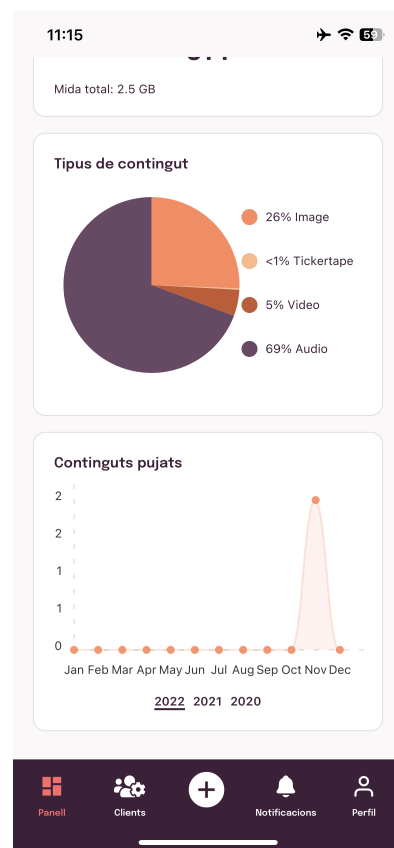
Figura 10.4: Navegació Bottom Tabs

10.2 Dashboard

L'objectiu de la vista de *Dashboard* és mostrar un resum general de dades importants i útils al *partner* sobre els seus clients. Cada bloc d'informació està encapsulada en una targeta.



(a) Captura Dashboard 1



(b) Captura Dashboard 2

Figura 10.5: Captures de pantalla de Dashboard

- **Localitzacions i *players*:** aquí es mostra el número de localitzacions i *players* en total dels clients.
- **Estat dels *players*:** es mostra el número de *players* en línia i en fora de línia.
- **Continguts:** el número de continguts i la mida en total.
- **Tipus de contingut:** gràfica circular amb els tipus de contingut.
- **Continguts pujats:** gràfica amb el nombre de continguts pujats per mesos. L'usuari pot alternar entre els últims 3 anys.

Crides a l'API

Descripció	Obtenir informació de la gràfica localitzacions & <i>players</i>
Mètode	GET
URL	/analytics/players/location
Paràmetres	idClient: ID del client
Respostes	200: { players: number; locations: number; } 401: Unauthorized 404: Not Found

Descripció	Obtenir informació de l'estat dels <i>players</i>
Mètode	GET
URL	/analytics/players/connections
Paràmetres	idClient: ID del client
Respostes	200: { now: { online: number, offline: number }; } 401: Unauthorized 404: Not Found

Descripció	Obtenir informació de la gràfica de continguts
Mètode	GET
URL	/analytics/contents
Paràmetres	idClient: ID del client
Respostes	200: { size: number; total: number; } 401: Unauthorized 404: Not Found

Descripció	Obtenir informació de la gràfica de tipus de continguts
Mètode	GET
URL	/analytics/contents/types
Paràmetres	idClient: ID del client
Respostes	200: { typeId: number; keyword: string; name: string; total: number; color: string; legendFontColor: string; legendFontSize: number; } 401: Unauthorized 404: Not Found

Descripció	Obtenir informació de la gràfica de continguts pujats
Mètode	GET
URL	/analytics/contents/uploads
Paràmetres	idClient: ID del client since: temps desde until: temps fins
Respostes	200: number[] 401: Unauthorized 404: Not Found

10.3 Clients

En aquesta secció es descriuen totes les vistes relacionades a gestió de clients, així com la llista de clients, localitzacions i *players* i la pròpia vista d'un *player*.

10.3.1 Llista de clients

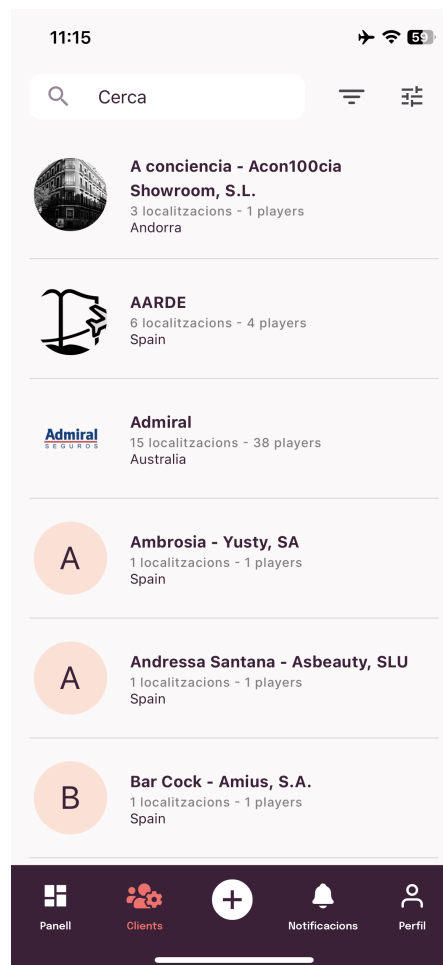


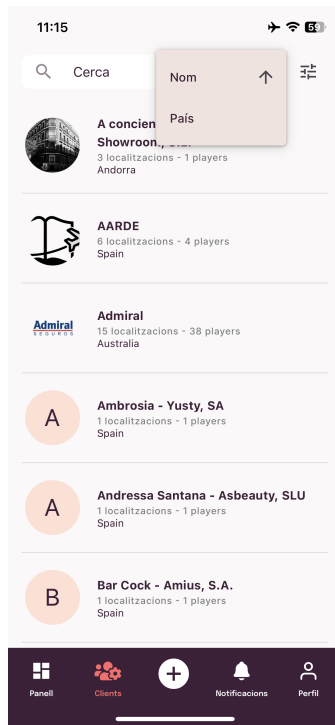
Figura 10.6: Llista de clients

El primer que es mostra en entrar a la pila de clients és la pròpia llista. Aquesta vista està formada per diferents components, com es pot veure a la figura 10.6.

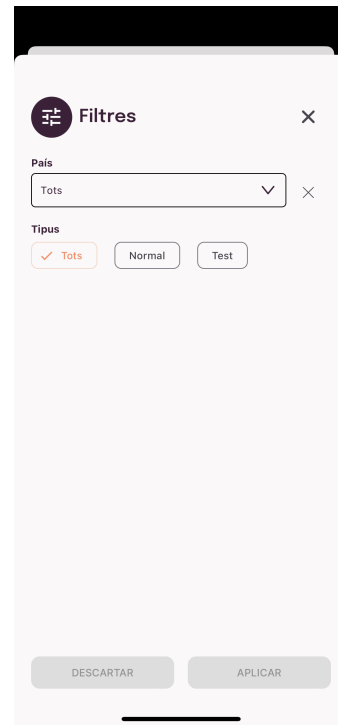
En primer lloc, la barra d'eines conté la barra de búsqueda per facilitar una cerca ràpida del client. Just al costat està els botons per ordenar la llista segons nom o país en ordre ascendent o descendent, i per filtrar segons diferents paràmetres.

En segon lloc, la pròpia llista és una llista virtualitzada, és a dir, es carrega de manera dinàmica i també va demanant informació al servidor quan es desplaça. Cada fila de la llista representa un client del *partner*; conté la foto de perfil, en cas de que hi tingui, el nom, el país i el nombre de localitzacions i *players* que té.

En la figura 10.7a es pot veure el *pop-up* que es mostra quan l'usuari clica a la icona d'ordenació. La fletxa indica el sentit de l'ordenació, ascendent o descendent. En la figura 10.7b podem visualitzar la vista de filtres quan l'usuari clica a la icona de filtres de la barra d'eines. Aquesta permet filtrar per país i el tipus de client.



(a) Pop-up ordenació



(b) Vista filtres

Figura 10.7: Barra d'eines clients

Crides a l'API

Descripció	Obtenir informació dels clients
Mètode	GET
URL	/clients
Paràmetres	<pre>{ page: number; take: number; order: { [order: string]: string; }; partner?: number; s?: string; filters?: { test?: boolean; country?: string; }; }</pre>
Respostes	200: <pre>{ data: Client[]; page: number; recordsTotal: number; totalPages: number; s?: string; }</pre> 401: Unauthorized 404: Not Found

10.3.2 Llista de localitzacions i *players* d'un client

En la llista de clients anterior, si l'usuari clica a un client, es navega a la vista de localitzacions i *players* d'aquest client. 10.8

En primer lloc, la capçalera conté tres elements: botó per tornar a la vista anterior (llista de clients), el nom del client que ha seleccionat i una icona que indica la configuració de notificacions per aquest client. Més endavant entrarem amb més detall amb les notificacions *push*.

En segon lloc, la barra d'eines és semblant al que hem vist anteriorment: conté la barra de cerca i el botó de filtres.

Per últim, la llista de localitzacions i *players*, que també és virtualitzada. La diferència principal d'aquesta és que els diferents *players* formen part d'una mateixa localització, per tant, es mostra una sub-capçalera amb el nom de la localització.

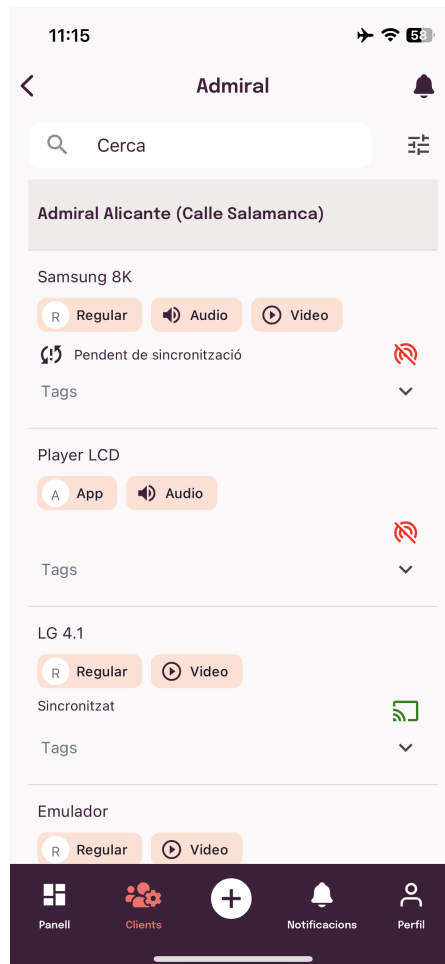


Figura 10.8: Llista de localitzacions i *players*

Element fila *player*

Cada fila representa un *player*. Aquesta conté el nom del *player*, el tipus i característiques, l'estat de sincronització, estat de connexió i un desplegable que mostra les etiquetes que té, en cas de que hi tingui.

En la figura 10.9 es pot veure el *pop-up* que es mostra quan l'usuari clica sobre la icona de l'estat de connexió. La icona en color vermell indica que està fora de línia, en canvi el verd indica que està en línia. En la finestra emergent es mostra la última IP del *player*, així com l'espai total i lliure que disposa. També es mostra

l'última connexió; es mostra el temps en diferents zones horàries, en el cas que l'usuari tingui una zona horària diferent a la de la localització.

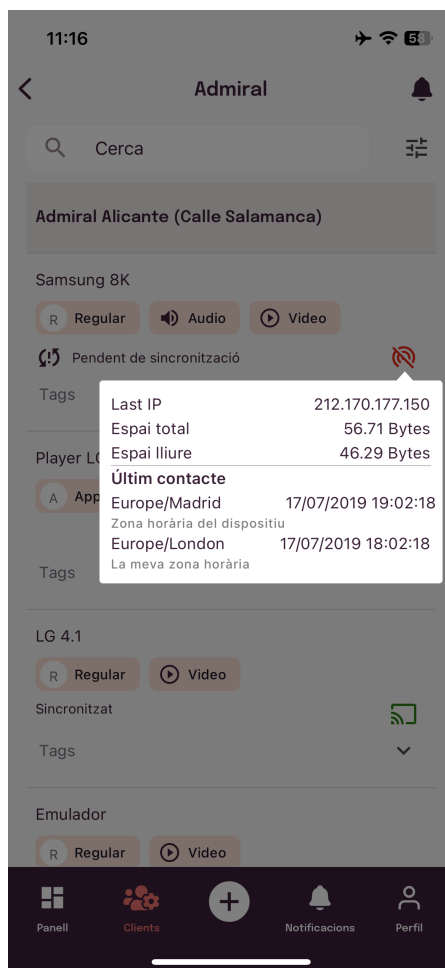
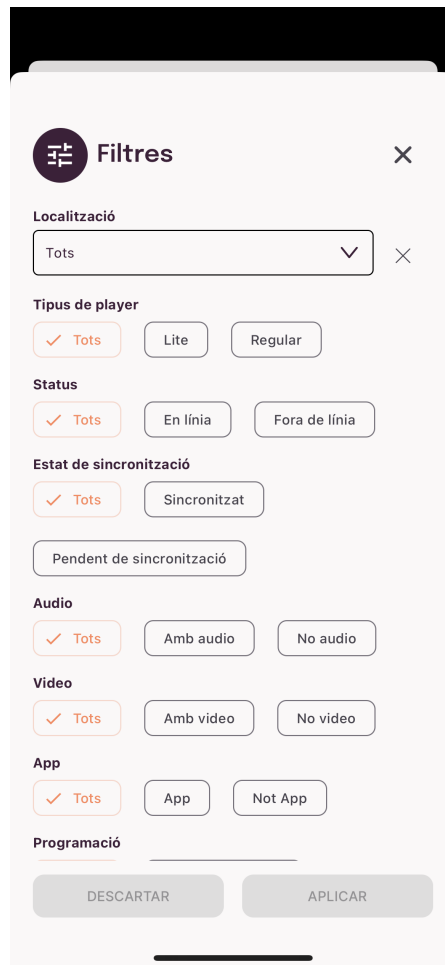


Figura 10.9: Estat de connexió d'un *player*

Filtres

En la figura 10.10 es pot veure la vista de filtres. En aquest cas, hi ha més varietat d'opcions: es pot filtrar per localització, per tipus de *player*, per estat de sincronització, per estat de connexió, si és només àudio, si és només vídeo, si és tipus 'App' i per programació.



Filtres ✕

Localització

Tots ▼ ✕

Tipus de player

✓ Tots Lite Regular

Status

✓ Tots En línia Fora de línia

Estat de sincronització

✓ Tots Sincronitzat

Pendent de sincronització

Audio

✓ Tots Amb audio No audio

Video

✓ Tots Amb video No video

App

✓ Tots App Not App

Programació

DESCARTAR APLICAR

Figura 10.10: Filtres de localitzacions i *players*

Crides a l'API

Descripció	Obtenir informació de localitzacions
Mètode	GET
URL	/locations
Cos de la petició	<pre>{ players: boolean; client?: number; s?: string; page?: number; take?: number; filters?: { location?: number; lite?: boolean; audio?: boolean; video?: boolean; app?: boolean; schedule?: boolean; status?: string; sync?: boolean; test?: boolean; }; }</pre>
Respostes	200: <pre>{ totalPages: number; recordsTotal: number; data: Location[]; page: number; }</pre> 401: Unauthorized 404: Not Found

10.3.3 Vista d'un *player*

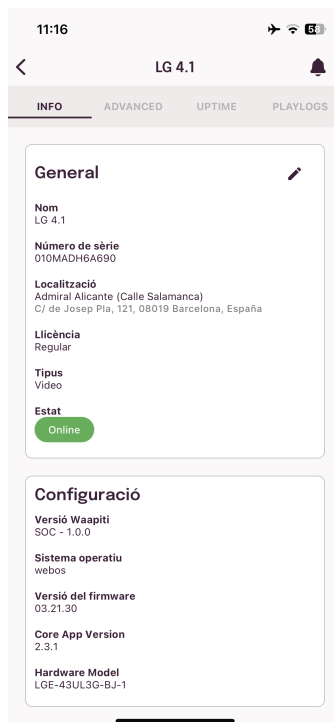
En aquesta subsecció es descriu la vista d'un *player*. És la vista més complexa i que conté més funcionalitats. En primer lloc, la capçalera és semblant a la llista anterior, amb la diferència que en aquest cas està el nom del *player*. La vista està organitzada utilitzant un navegador de pestanyes, cada pestanya correspon a una

categoria.

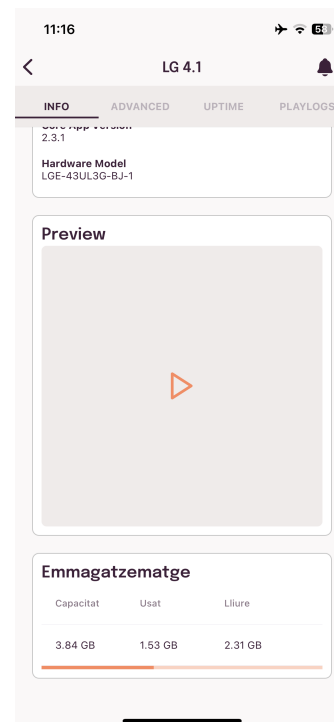
INFO

En aquesta pestanya es mostra una informació general del *player*, així com característiques tècniques.

- **General:** es mostra informació general del *player*, com el nom, número de sèrie, la localització a la que pertany, la llicència, el tipus i l'estat de connexió. A dalt a la dreta hi ha un botó que editar aquesta informació, més endavant es detallarà.
- **Configuració:** es mostra informació sobre la configuració del *player*, així com característiques més tècniques.
- **Preview:** aquesta targeta només es mostra si el *player* està en línia. Si l'usuari clica en el botó de reproduir, aquesta comença a mostrar el contingut que s'està reproduint en temps real.
- **Emmagatzematge:** informació sobre l'espai total i lliure.



(a) Targetes general i configuració



(b) Targetes preview i emmagatzematge

Figura 10.11: Captures de pantalla de la pestanya 'INFO'

Com s'ha comentat anteriorment, en la figura 10.12 es veu un exemple de visualitzar el contingut que s'està reproduint en temps real. Aquesta s'adapta segons la resolució i orientació de la pantalla; en aquest cas, és una pantalla que està amb orientació vertical i resolució Full HD. Aquesta funcionalitat es connecta via *socket* [25] amb el servidor i mostra el contingut segons els missatges que aquest envia.

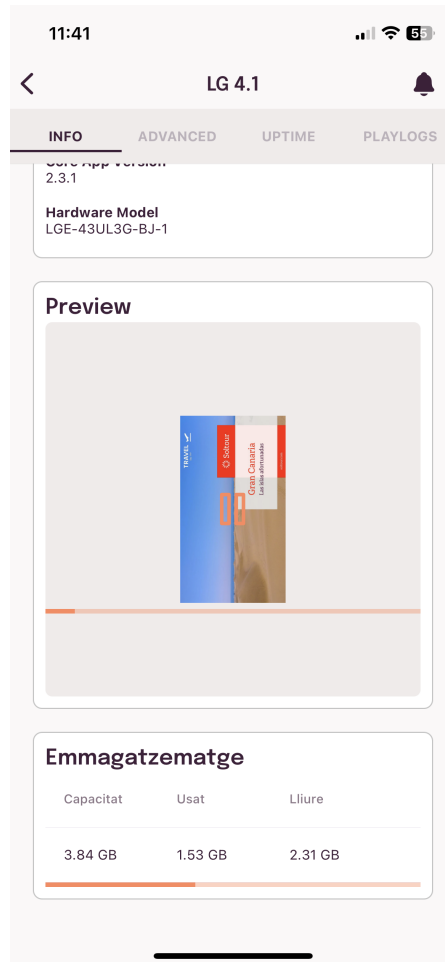


Figura 10.12: Preview d'un *player*

En la figura 10.13 es mostra la vista d'editar la informació d'un *player*.

Device information ✕

Nom

Llicència

Localització
 ▼

Tipus de player
 ▼

Figura 10.13: Editar informació d'un *player*

Crides a l'API

Descripció	Obtenir informació d'un <i>player</i>
Mètode	GET
URL	/players/:playerId
Paràmetres	playerId: ID del <i>player</i>
Respostes	200: Player 401: Unauthorized 404: Not Found

Descripció	Editar <i>player</i>
Mètode	PUT
URL	/players/:playerId
Cos de la petició	<pre>{ name?: string; locationId?: number; lite?: boolean; audio?: boolean; video?: boolean; app?: boolean; }</pre>
Respostes	200: Player 401: Unauthorized 404: Not Found

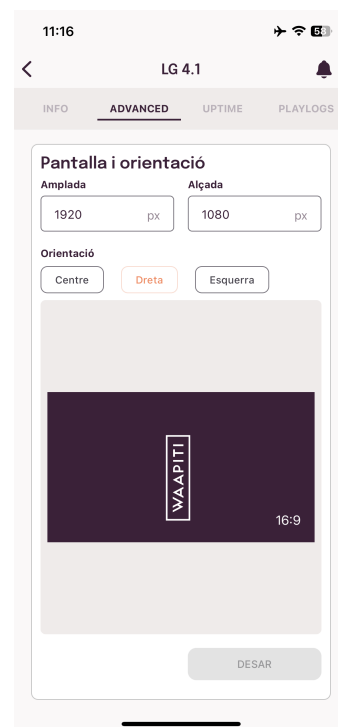
ADVANCED

En aquesta pestanya es mostra informació avançada del *player*.

En primer lloc, en la figura 10.14a podem veure les principals accions que podem realitzar, com: apagar la pantalla, reiniciar reproducció, reiniciar el *player* o forçar sincronització. També podem canviar de canal. En la figura 10.14b podem editar la configuració de pantalla i orientació.



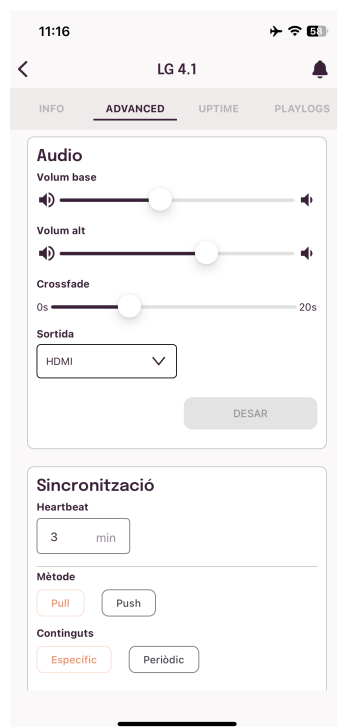
(a) Targeta accions



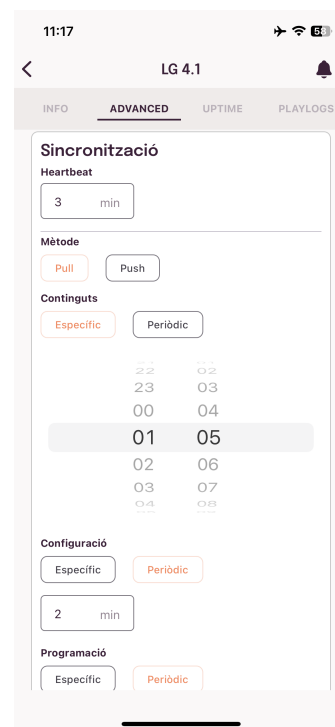
(b) Targeta pantalla i orientació

Figura 10.14: Captures de pantalla de la pestanya 'ADVANCED'

Seguidament, en les figures de 10.15 podem veure les targetes per configurar l'àudio i la sincronització.



(a) Targeta audio



(b) Targeta sincronització

Figura 10.15: Captures de pantalla de la pestanya 'ADVANCED'

Crides a l'API

Descripció	Obtenir informació avançada d'un <i>player</i>
Mètode	GET
URL	/players/:playerId/settings
Respostes	200 <pre>{ heartbeatPeriod: string; sync: { push: boolean; contentsMethod?: string; contentsPeriod: string; settingsMethod?: string; settingsPeriod: string; schedulesMethod?: string; schedulesPeriod: string; }; audio: { baseVolume: number; crossfade: number; highVolume: number; output: number; }; video: { height: number; width: number; orientation: number; }; }</pre> 401: Unauthorized 404: Not Found

Descripció	Editar informació avançada d'un <i>player</i>
Mètode	PUT
URL	/players/:playerId/settings
Respostes	<pre> { heartbeatPeriod: string; sync: { push: boolean; contentsMethod?: string; contentsPeriod: string; settingsMethod?: string; settingsPeriod: string; schedulesMethod?: string; schedulesPeriod: string; }; audio: { baseVolume: number; crossfade: number; highVolume: number; output: number; }; video: { height: number; width: number; orientation: number; }; }</pre>
Respostes	200 401: Unauthorized 404: Not Found

Descripció	Executar accions de <i>player</i>
Mètode	PUT
URL	/players/:playerId/control
Cos de la petició	<pre>{ action: string; value?: boolean; }</pre>
Respostes	200 401: Unauthorized 404: Not Found

UPTIME

Aquí es mostra el temps de connexió del *player* en hores. Inclou un calendari per facilitar la navegació. El color verd indica que està en línia i el vermell que no ho està. Cal destacar que el temps es mostra en la zona horària de l'usuari; en aquest cas 10.16 es mostra en Londres.

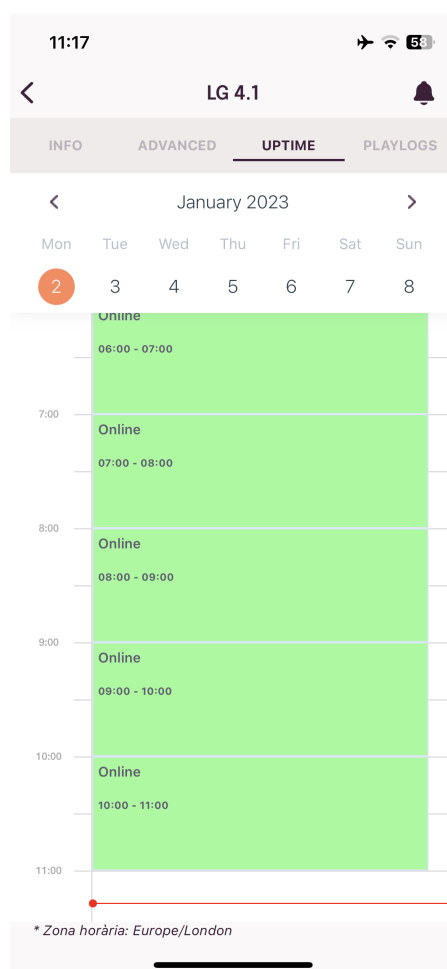


Figura 10.16: Temps de connexió d'un *player*

Crides a l'API

Descripció	Obtenir informació en línia d'un <i>player</i>
Mètode	GET
URL	/players/:playerId/uptime
Cos de la petició	<pre>{ since: string; until: string; }</pre>
Respostes	200 <pre>{ deviceId: number; time: string; online: boolean; }</pre> 401: Unauthorized 404: Not Found

PLAYLOGS

Aquesta última pestanya mostra un historial de reproducció del *player*. A més, es connecta també via *socket* amb el servidor i mostra logs en temps real.

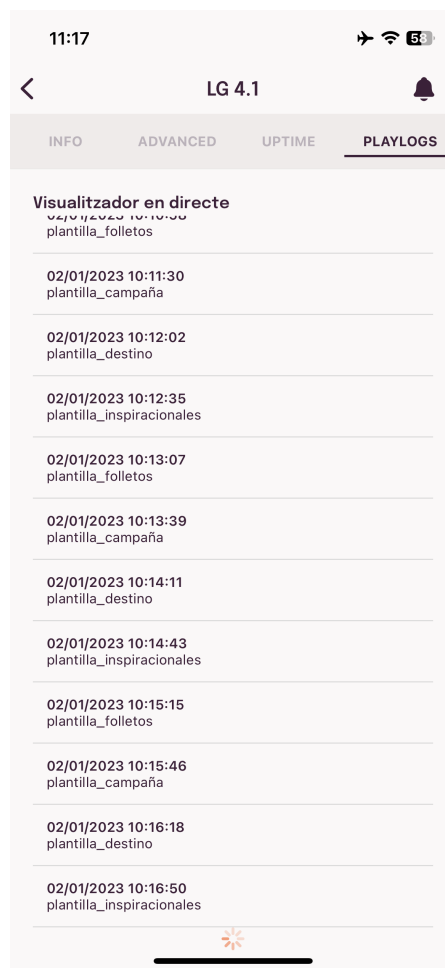


Figura 10.17: Playlogs d'un *player*

Crides a l'API

Descripció	Obtenir informació de logs d'un <i>player</i>
Mètode	GET
URL	/players/:playerId/playlogs
Cos de la petició	<pre>{ playerId: number; count: number; }</pre>
Respostes	200 <pre>{ timestamp: number; value: string; }</pre> 401: Unauthorized 404: Not Found

10.4 Registrar nou *Player*

Com s'ha comentat en anteriors capítols, una de les funcionalitats principals del projecte és poder oferir la possibilitat de registrar nous *players* a la plataforma de Waapiti escanejant codi QR. Aquesta funcionalitat és molt útil per a *partners* ja que disminuïm la fricció i el temps de registre.

Per tal d'explicar el funcionament de registre, simularem un cas real amb un *partner* i client imaginari. El *partner* es diu 'Partner Longbo', el client al qual volem registrar és 'Hotel Longbo' i la localització serà 'Recepció'. Cal destacar que en aquest apartat, el terme 'pantalla' es refereix al suport audiovisual al qual estem registrant el *player*.

Una vegada presentat els protagonistes, es detalla el procés amb una seqüència de captures de pantalla, tant per part de l'aplicació com per la pantalla el qual el *player* està reproduïnt.

Quan s'encén el *player*, es mostra una pantalla de benvinguda com es pot veure en 10.18. Per part de l'aplicació, la vista principal al clicar la icona de més en la barra de navegació és la que es mostra en 10.19, abaix de la finestra de càmera es mostra un llistat d'instruccions. En cas que el codi QR sigui invàlid o es fagi un ús inadequat es mostra una alerta amb l'error corresponent. Per tant, el *partner* apuntaria la càmera per tal de quadrar el codi QR i escanejar-lo.

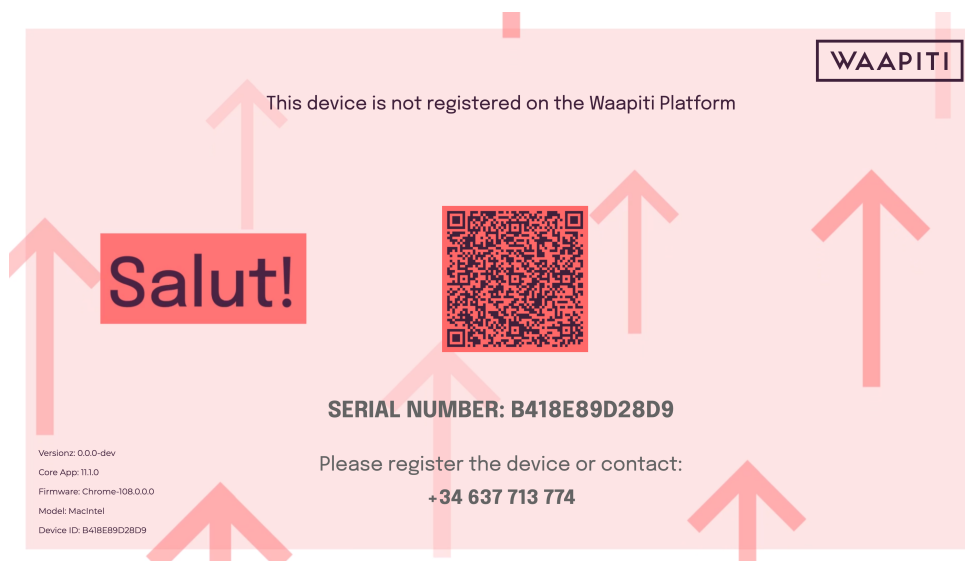


Figura 10.18: Vista pantalla: benvinguda

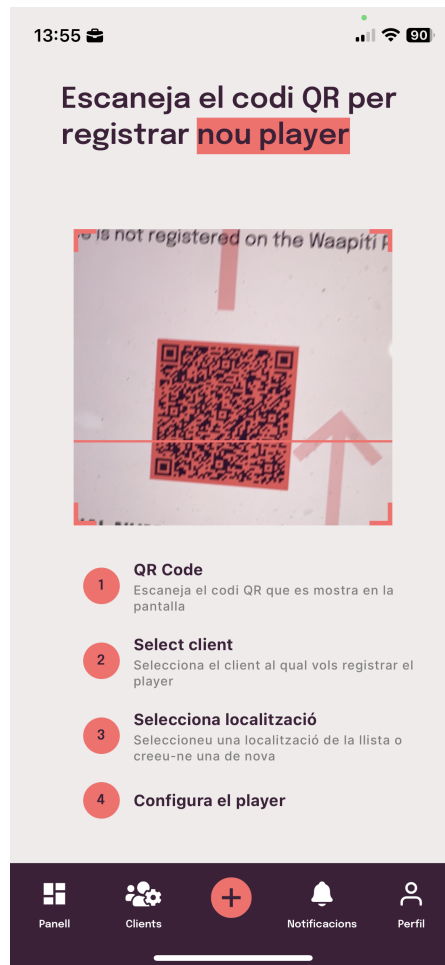


Figura 10.19: Vista principal per registrar nou *player*

En cas que el codi QR sigui vàlid, immediatament, en la pantalla es mostra animació informant que s'està iniciant el sistema, com en la figura 10.20

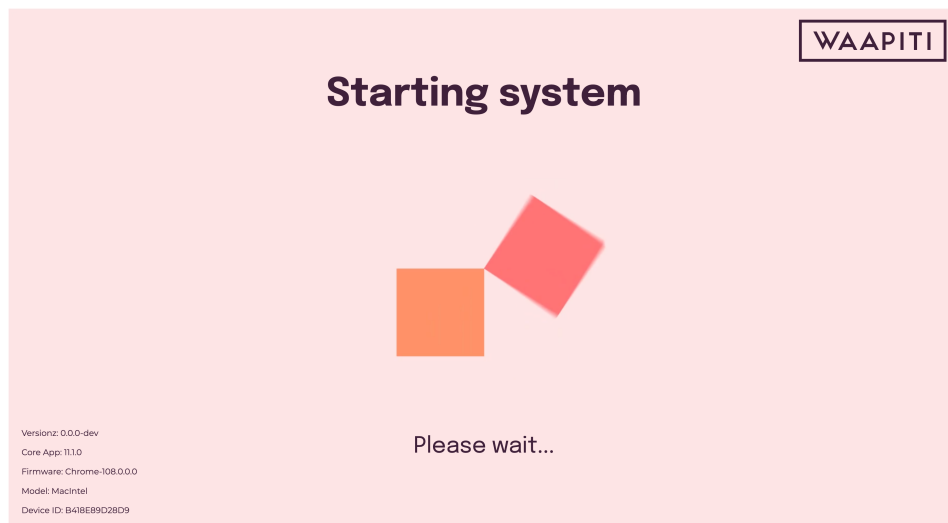


Figura 10.20: Vista pantalla: iniciant sistema

Després de que el sistema s'hagi iniciat, es mostra informació sobre el partner, com es pot veure en la figura 10.21

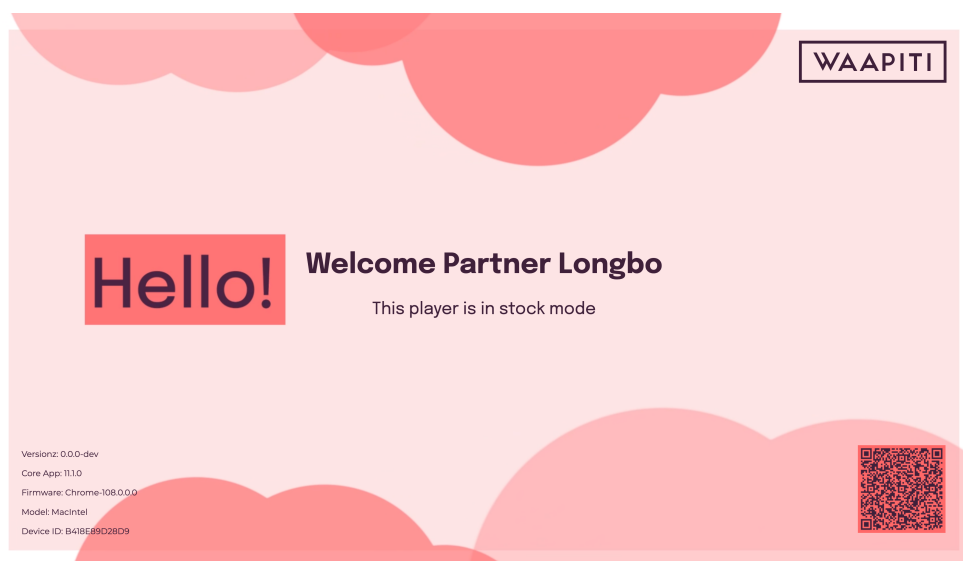
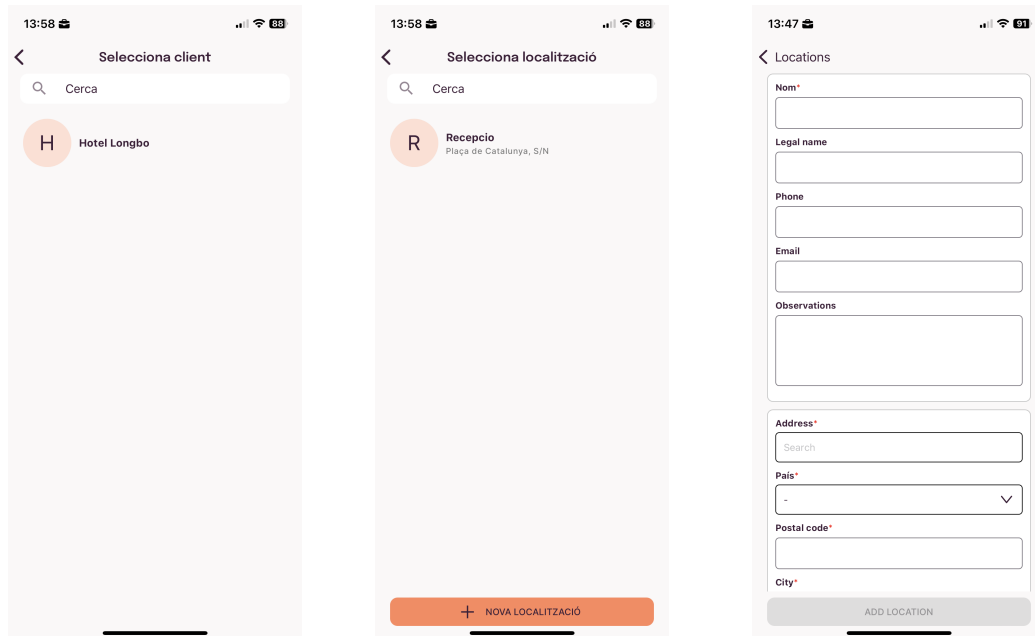


Figura 10.21: Vista pantalla: sistema iniciat

Per altra banda, en l'aplicació, l'usuari ha de seleccionar el client al qual vol registrar aquest dispositiu, així com la seva localització. En la vista de seleccionar

una localització, també s'ofereix la opció de crear-ne una de nova, com es pot veure en les figures 10.22b i 10.22c.



(a) Seleccionar client

(b) Seleccionar localització

(c) Afegir nova localització

Figura 10.22: Procés seleccionar client i localització

Simultàneament, una vegada escollida les opcions, a la pantalla es mostra la informació amb la localització 10.23.



Figura 10.23: Vista pantalla: localització

Per últim, queda el pas de configurar el tipus i llicència del *player* i donar-li a desar i tindriem el *player* registrat 10.24. L'usuari sempre pot canviar la localització o el client escollit abans de guardar-ho.

13:58

Configura el player

H Hotel Longbo

R Recepcio
Plaça de Catalunya, S/N

Tap to change

Nom*

Nou dispositiu

Llicència*

Regular

Tipus de player*

Video

DESAR

Figura 10.24: Configurar *player*

Un punt a destacar és que, en el cas que durant el procés de registre l'usuari apaga la pantalla per qualsevol motiu o es queda sense connexió a Internet, es guarda l'estat del registre per tal de poder continuar-lo més tard. Com es pot veure en les figures 10.21 i 10.23, abaix a la dreta hi ha un codi QR. Això implica que si l'usuari escaneja aquest codi, es mostra en el pas on s'ha quedat prèviament.

Crides a l'API

Cal mencionar que per dur a terme aquesta funcionalitat, s'ha hagut de crear nous endpoints en el servidor de l'empresa. La tasca ho ha realitzat el director d'aquest projecte.

Descripció	Comprovar estat codi QR
Mètode	POST
URL	/players/qr-token
Cos de la petició	token: string
Respostes	200 { id: number; serial: string; status: 'stock' OR 'unlicensed'; location: Location; } 401: Unauthorized 404: Not Found

Descripció	Registrar <i>player</i>
Mètode	PUT
URL	/players/:playerId
Cos de la petició	{ name: string; locationId?: number; serialNumber: string; lite: boolean; audio: boolean; video: boolean; app: boolean; clientId?: number; }
Respostes	200: Player 401: Unauthorized 404: Not Found

Descripció	Assignar nova localització a un <i>player</i>
Mètode	PUT
URL	/players/:playerId/location
Cos de la petició	locationId: number
Respostes	200 401: Unauthorized 404: Not Found

10.5 Notificacions

Aquesta vista mostra un historial de totes les notificacions rebudes. Cada element conté el tipus de notificació, el *player* i la localització al qual pertany i el moment en que es va rebre. Si es fa clic en un element, es navega a la vista del *player* corresponent.

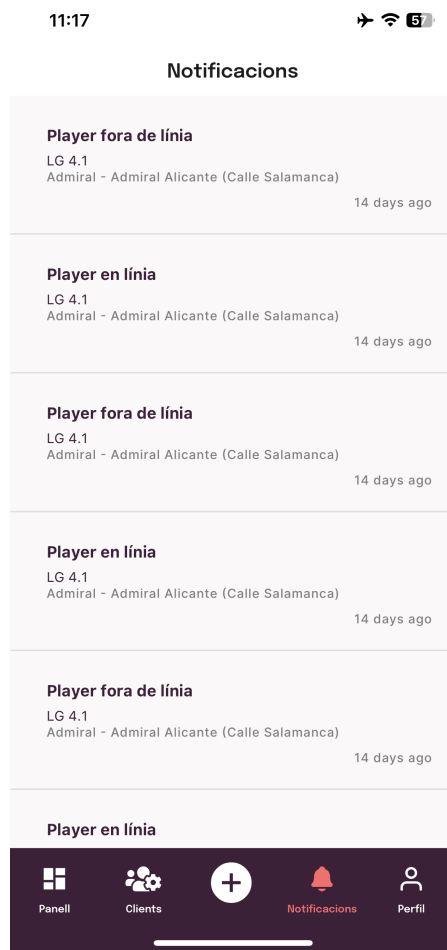


Figura 10.25: Vista principal per registrar nou *player*

En el cas que l'usuari tingui notificacions sense llegir, es mostra un indicador amb el nombre de notificacions pendents a la barra de navegació.

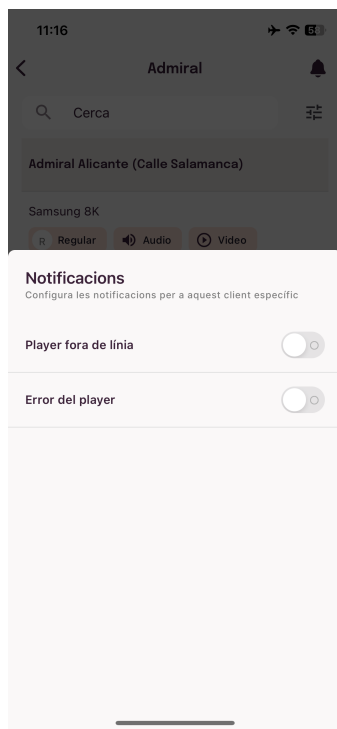


Figura 10.26: Navegació Bottom Tabs

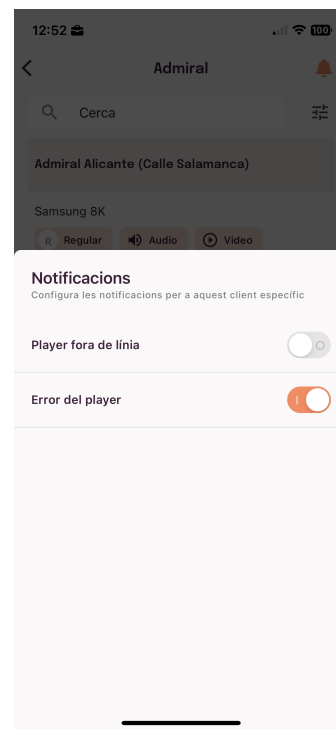
També aprofitaré per detallar com es configura el sistema de notificacions. L'aplicació té 3 capes de nivell de notificació: general, segons client i segons *player*. Com

hem vist en anteriors captures, a la capçalera hi havia una icona de notificació; si es clica en aquesta, es mostra un modal per configurar les notificacions, com es pot veure en 10.27a.

Si hi ha alguna configuració activa, la icona de notificació es mostra d'un color diferent, com es pot veure en 10.27b.



(a) Notificacions d'un client



(b) Notificacions client actives

Figura 10.27: Captures de pantalla del modal de notificacions

10.5.1 Implementació sistema notificacions

Un dels objectius principals del projecte és proporcionar avisos i incidències a l'instant mitjançant alertes notificacions push, d'aquesta manera es mantindrà informat en tot moment. Per poder implementar aquesta funcionalitat és necessari utilitzar un servei de missatgeria per cada plataforma: *Firebase Cloud Messaging (FCM)* per dispositius Android i *Apple Push Notification Service (APNS)* per dispositius iOS. Així mateix es necessita un servei de notificacions que es comuniqui amb aquestes. Com a conseqüència es va fer un estudi sobre diferents alternatives: *Expo Push API* i *AWS SNS*.

- **Expo Push API:** Expo Push API és un servei que facilita la implementació de les notificacions push ja que es delega tota la configuració al servei.

Avantatges	Desavantatges
Fàcil d'implementar, tant frontend com backend	Car
Documentació completa	

Taula 10.1: Avantatges Expo Push API

Font: Elaboració pròpia

- **AWS SNS:** Amazon Simple Notification Service (Amazon SNS) és un servei administrat el qual ofereix el lliurament de missatges dels publicadors als subscriptors (també conegut com a productors i consumidors). Els publicadors es comuniquen de manera asíncrona amb els subscriptors mitjançant l'enviament de missatges a un tema, que és un punt d'accés lògic i un canal de comunicació. Els clients es poden subscriure al canal de SNS i rebre missatges publicats mitjançant un tipus de punt d'enllaç compatible.

Avantatges	Desavantatges
Gratuït fins a 1M de missatges	Més complicat d'implementar
	Es necessita certificats per les dues plataformes

Taula 10.2: AWS SNS

Font: Elaboració pròpia

Al final es va decidir utilitzar *AWS SNS* pel cost en comparació amb *Expo Push API*. A més, *AWS SNS* és més escalable i pot ser utilitzat per altres serveis.

En la figura 10.28 podem veure un exemple de notificacions *push* en acció. Si l'usuari clica sobre la notificació, aquest redirigeix a la vista del *player* corresponent.

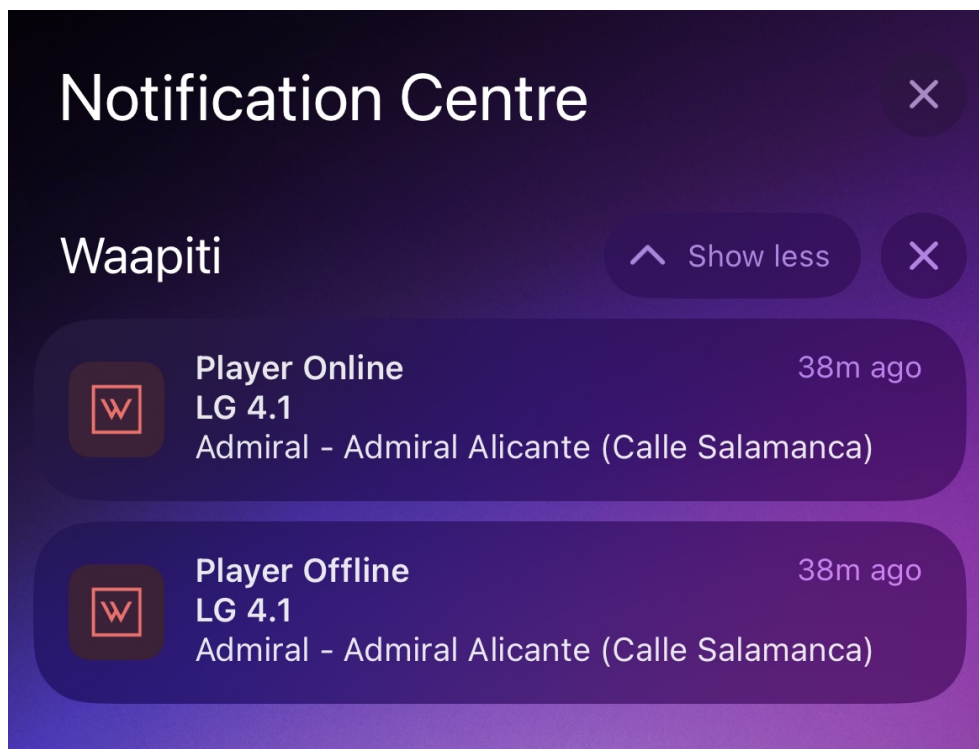


Figura 10.28: Exemple de notificacions push

Per tal de que funcioni correctament el sistema de notificacions en la part front-end, primer necessitem l'anomenat *Device push token*, que essencialment és un identificador únic per aplicació-dispositiu, proporcionat per Google o Apple, utilitzat per la passarel·la de notificacions *push*. Una vegada obtingut aquest *token*, ho enviem al servidor per guardar-ho i així tenir registrat el dispositiu. Si l'usuari té el permís de notificacions desactivat, no es pot obtenir el *token*; s'avisarà a l'usuari amb una alerta.

En segon lloc, per rebre les notificacions s'utilitza la llibreria *expo-notifications* el qual ofereix una API per a la gestió de notificacions. Una vegada que l'aplicació s'inicia, després de registrar el *token*, s'inicialitza un *listener* que estarà escoltant tots els esdeveniments de notificacions que pot rebre.

Per últim, respecte a la interacció amb les notificacions, com per exemple obrir la vista corresponent a les dades de la notificació, s'ha hagut d'implementar abans un sistema de *deep linking*. En altres paraules, crear un enllaç entre les diferents vistes i piles. D'aquesta manera, tenim control total sobre la redirecció de les vistes.

En la figura 10.29 podem veure una representació de com funciona aquest sistema.

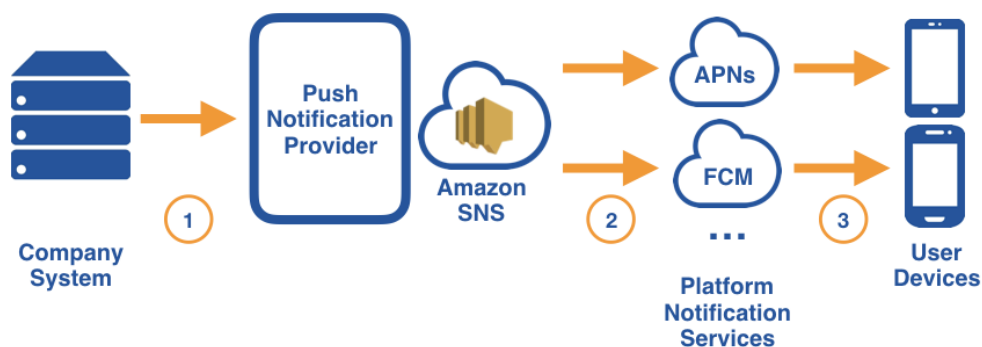


Figura 10.29: Diagrama sistema de notifiacions *push*
Font: ArterKirkwood

Crides a l'API

Per aquesta funcionalitat també s'ha hagut d'implementar nous endpoints en el back-end.

Descripció	Obtenir notifiacions
Mètode	GET
URL	/account/notifications
Paràmetres	count: number lastKey: number
Respostes	200 <pre>{ notifications: AccountNotification[]; unread: number; lastKey: number; }</pre> 401: Unauthorized 404: Not Found

Descripció	Marcar notifikacions com a llegides
Mètode	POST
URL	/account/notifications/read
Cos de la petició	lastKey: number
Respostes	200 401: Unauthorized 404: Not Found

Descripció	Guardar device push <i>token</i>
Mètode	POST
URL	/notifications/token
Cos de la petició	token:string platform:string
Respostes	200 401: Unauthorized 404: Not Found

10.6 Perfil

En aquesta vista 10.30 es mostra la foto de perfil de l'usuari, botó per editar el perfil, navegar a la configuració de notifikacions generals, navegar a la vista de sobre Waapiti i, per últim, el botó per tancar la sessió.

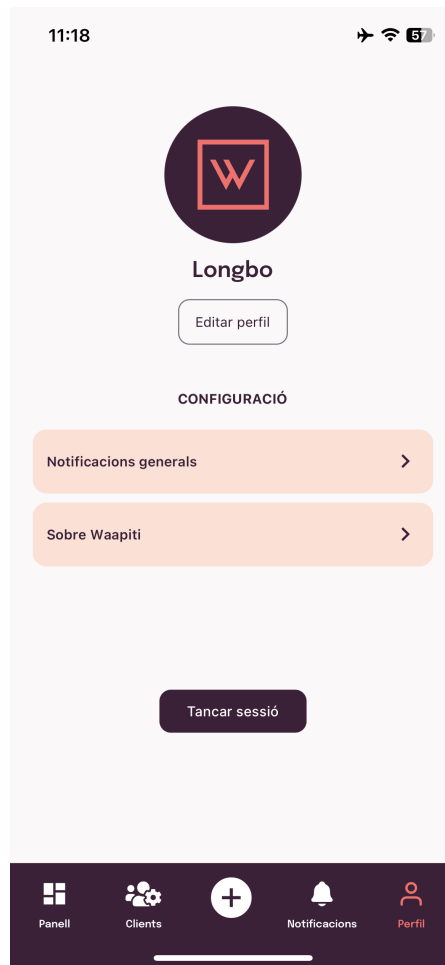
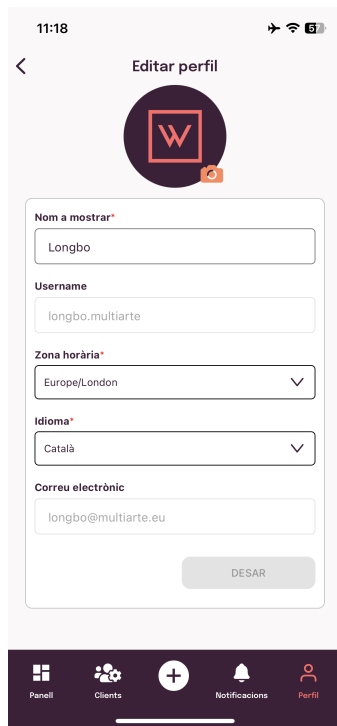
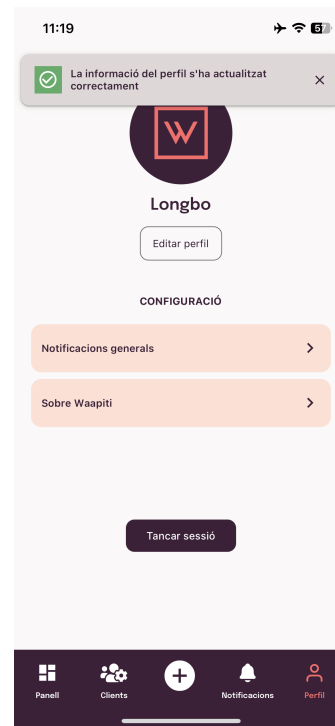


Figura 10.30: Vista principal de perfil

En editar el perfil, l'usuari pot canviar la seva imatge de perfil, el nom a mostrar, la zona horària o el seu idioma preferit. També es permet canviar la contrasenya, que està situat més abaix de la resta de camps. En cas d'editar correctament la informació, es mostra una alerta com es veu en 10.31b.



(a) Editar perfil



(b) Alerta de canvi correcte

Figura 10.31: Captures d'editar perfil

Per últim, la vista de sobre Waapiti 10.32 mostra informació sobre l'empresa, així com el botó per enviar un correu electrònic a l'equip, condicions d'ús i política de privacitat. També es mostra la versió de l'aplicació, per si es vol reportar algun error.



Figura 10.32: Vista sobre Waapiti

Crides a l'API

Descripció	Editar perfil
Mètode	PUT
URL	/account
Cos de la petició	{ name?: string; language?: string; timezone?: string; }
Respostes	200 401: Unauthorized 404: Not Found

Descripció	Canviar contrasenya
Mètode	PUT
URL	/account/password
Cos de la petició	{ oldPassword: string; password: string; passwordConfirmation: string; }
Respostes	200 401: Unauthorized 404: Not Found

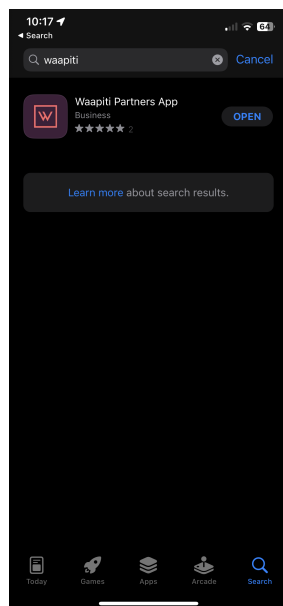
Capítol 11

Instal·lació i execució

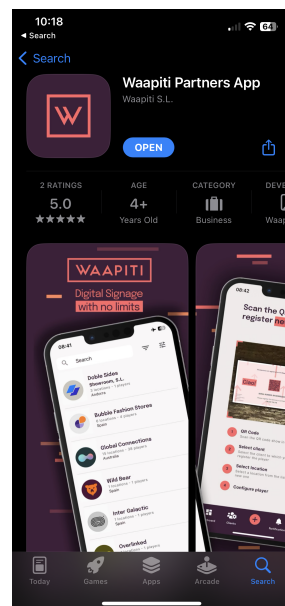
11.1 iOS

Per a usuaris amb dispositius iOS, l'aplicació es pot descarregar des de l'App Store. Es necessita un dispositiu amb iOS 13 o superior.

Link: <https://apps.apple.com/es/app/waapiti-partners-app/id644465661>



(a) Búsqueda App Store



(b) Vista aplicació

Figura 11.1: Descàrrega de l'aplicació iOS

11.2 Android

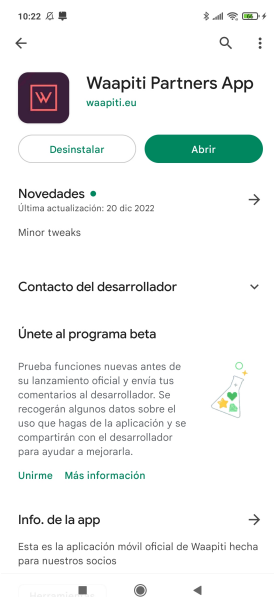
Per a usuaris amb dispositius Android, l'aplicació es pot descarregar des de la *Google Play Store*. D'aquesta manera, l'usuari pot instal·lar l'aplicació fàcilment i rebre actualitzacions automàtiques en segon pla.

Requereix un dispositiu amb Android 5.0 o superior.

Link: <https://play.google.com/store/apps/details?id=com.waapiti.partnersapp>



(a) Búsqueda Google Play



(b) Vista aplicació

Figura 11.2: Descàrrega de l'aplicació Android

Capítol 12

Proves

En aquest capítol es descriuen les proves que s’han realitzat per a la validació del projecte.

12.1 Proves unitàries

Les proves unitàries són proves de software amb l’objectiu de comprovar que cada mòdul del programa funciona correctament. Aquestes ens permeten aïllar els diferents mòduls. Com el projecte utilitzem React Native, les proves unitàries s’han considerat a nivell de component de React.

Aquestes proves s’han realitzat amb la llibreria *Jest*. És la llibreria recomanada per la pròpia documentació de React. El que es fa és simular un context amb dades constants i es testeja un mòdul, d’aquesta manera, al executar una prova, es compara la seva sortida amb la sortida esperada.

12.2 Proves d’integració

Les proves d’integració són proves de software amb l’objectiu de comprovar el comportament correcte entre diferents mòduls del programa.

La majoria de proves han estat de tipus unitàries, però també s’han realitzat proves d’integració a nivell de components de tipus vista i controlador, així com la interacció amb elements de l’arquitectura Flux.

Cal destacar que no ha estat possible realitzar proves la navegació entre diferents vistes degut a la limitació de la llibreria *React Navigation*, ja que compila a codi natiu i no és possible fer proves amb la llibreria *Jest* (només per Javascript).

12.3 Proves internes

Abans de publicar l'aplicació a les plataformes de distribució, s'han realitzat proves internes per assegurar el correcte funcionament i també aconseguir retroacció per part dels *stakeholders*.

Respecte iOS, s'ha anat publicant diferents versions a *TestFlight* per a poder distribuir l'aplicació a un nombre limitat de persones. Aquest servei permet als usuaris enviar informes de bugs i també fer suggeriments directament des de l'aplicació.

D'altra banda, per Android, s'ha anat publicant versions de tipus *Open testing*. La funcionalitat és semblant a la de *TestFlight*, però aquesta està publicada directament a la *Play Store* i no cal que l'usuari descarregui una altra aplicació com a intermediari.

D'aquesta manera, hem aconseguit obtenir retroacció directe per part dels *stakeholders*.

12.4 Control de *crashes* en producció

Una de les avantatges que ens ofereix el panell de *Play Store* és el control d'errors sobre la nostra aplicació en producció. Aquesta plataforma ens ofereix estadístiques d'ús. A més, disposa d'un historial d'errors. Com es pot veure en les següents figures, ens especifica el nombre d'errors que s'han produït en un rang de dates, gràfica sobre els dispositius i, el més important, codi de l'error que ha provocat el *crash*.

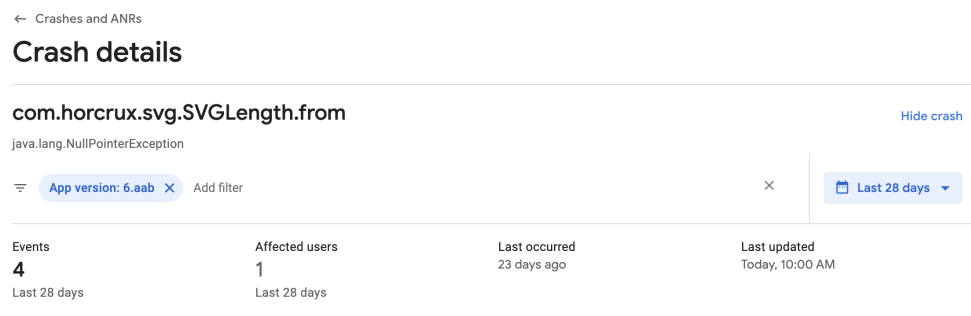


Figura 12.1: Nombre d'errors

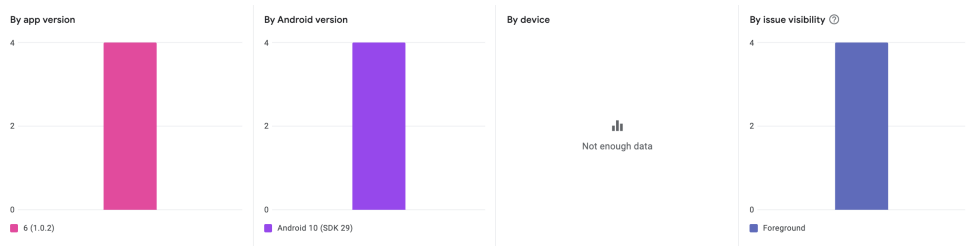


Figura 12.2: Gràfica sobre els dispositius

Crash details

Stack trace

1 < >

Type java.lang.NullPointerException

Show less ^

```
Exception java.lang.NullPointerException: Attempt to invoke virtual method 'double java.lang.Double.doubleValue()' on
at com.horcrux.svg.SVGLength.from (SVGLength.java:111)
at com.horcrux.svg.LinearGradientView.setX1 (LinearGradientView.java:53)
at com.horcrux.svg.RenderableViewManager$LinearGradientManager.setX1 (RenderableViewManager.java:1959)
at com.horcrux.svg.RenderableViewManager$LinearGradientManager.setX1 (RenderableViewManager.java:1904)
at com.facebook.react.viewmanagers.RNSVGLinearGradientManagerDelegate.setProperty (RNSVGLinearGradientManagerDelegat
at com.facebook.react.uimanager.ViewManagerPropertyUpdater.updateProps (ViewManagerPropertyUpdater.java:46)
at com.facebook.react.uimanager.ViewManager.updateProperties (ViewManager.java:84)
at com.facebook.react.uimanager.NativeViewHierarchyManager.updateProperties (NativeViewHierarchyManager.java:142)
at com.facebook.react.uimanager.UIImplementation.synchronouslyUpdateViewOnUIThread (UIImplementation.java:306)
at com.facebook.react.uimanager.UIManagerModule.synchronouslyUpdateViewOnUIThread (UIManagerModule.java:402)
at com.facebook.react.animated.PropsAnimatedNode.restoreDefaultValues (PropsAnimatedNode.java:90)
at com.facebook.react.animated.NativeAnimatedNodesManager.restoreDefaultValues (NativeAnimatedNodesManager.java:500)
at com.facebook.react.animated.NativeAnimatedModule$20.execute (NativeAnimatedModule.java:910)
at com.facebook.react.animated.NativeAnimatedModule$ConcurrentOperationQueue.executeBatch (NativeAnimatedModule.java
```

Figura 12.3: Codi de l'error